



简介：软件验证、确认和测试

测试背景

目的



目的

定义常见的测试术语

测试背景



历史



| 鉴于软件开发问题日益增多，“软件工程”一词在1968年在西德召开的一次研讨会上首次使用。

- 高成本
- 难以管理
- 可靠性低
- 缺乏用户接受度
- 难以维护

软件开发的当前状况



高成本

难以管理

可靠性低

缺乏用户接受度

难以维护

可靠性低



| 软件缺陷率大概在每一千条代码中有1个交付缺陷。

| 由于应用程序涵盖数百万条代码，所以客户会经历到很多缺陷。

定义



| 可靠性：

- 一个软件程序在给定的运行时间段内、而不出现软件错误的概率

测试人员与民意调查员的比较



定义



| 确认：

- 我们打造的产品是用户想要的吗？

| 验证：

- 我们打造的产品是正确（满足功能要求）的吗？

| 测试：

- 通过在样本数据集上执行程序来检查程序的行为

定义



| 错误：

- 人类犯的错误

| 缺陷/故障：

- 代码中显示的过失结果

| 失败：

- 软件无法达至原定成效

总结





简介：软件验证、确认和测试

测试完整生命周期

目的



目的

描述如何将测试整合到软件开发阶段中



目的

定义不同级别测试的目的

测试完整生命周期



| 瀑布

| 敏捷

| 测试驱动开发

| 敏捷方法：敏捷测试完整指南

敏捷测试



| 持续整合（至少每天）

- 静态代码分析
- 编译
- 单元测试
- 部署到测试环境中
- 整合/回归测试

测试驱动开发（红/绿/重构周期）



| 红色阶段：

- 对所需的行为，编写最低程度的测试

| 绿色阶段：

- 仅编写必要的代码使其能够通过最低程度的测试

| 重构阶段：

- 让测试保持绿色，同时改进代码

软件开发过程与测试开发过程的比较



测试等级



单元/组件

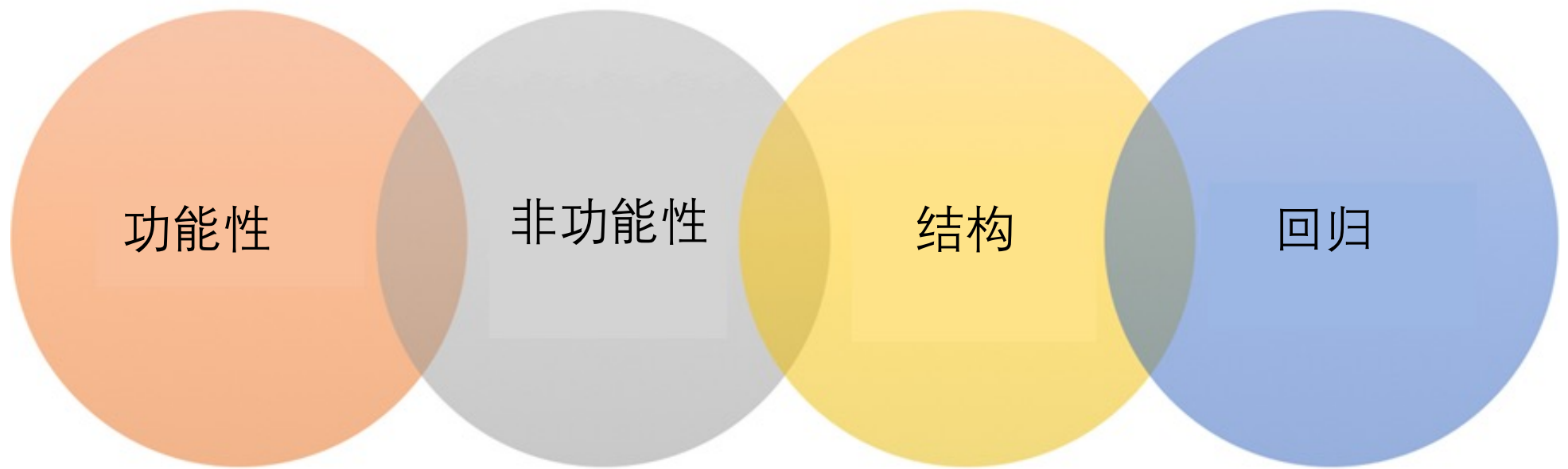
整合

系统

接受度

试用

测试类型



总结





简介：软件验证、确认和测试

测试原则和最佳实践

目的



目标

说明软件测试的最佳实践方法

测试原则



| 原则一：

- 测试仅能显示缺陷的存在 — 而非用以证明正确

| 原则二：

- 穷举测试是不可能的

| 原则三：

- 尽早开始测试

测试原则



| 原则一：

- 测试仅能显示有存在缺陷 — 而非用以证明正确

| 原则二：

- 穷举测试是不可能的

| 原则三：

- 尽早开始测试

| 原则四：

- 缺陷群集

| 原则五：

- 测试取决于上下文

| 原则六：

- 无错误谬论

测试态度



测试中犯的典型错误



| 坚信系统测试的首要目的是寻找错误

- 测试必须集中于发现重要问题
- 测试必须提供系统质量的评估

| 不关注使用性问题

| 开始太晚

- 测试必须帮助开发人员规避问题

典型的测试误解



| 把压力和性能测试推迟到最后

| 不测试文档

| 未在测试团队中纳入领域专家

| 未与开发人员进行良好的沟通

| 未能充分记录和审查测试设计

典型的测试误解



没能保持测试计划的灵活性



没能从之前的测试活动中汲取经验

最佳测试实践



| 通过统计测试，评估软件的可靠性

| 开发敏捷测试设计

- 适应后期更改
- 加强回归测试

| 利用基于模型的测试技术
- 状态图

| 培养跨功能的开发与测试团队

最佳测试实践



在可能的情况下，
自动生成测试



加强使用性测
试

ISO/IEC/IEEE 29119 软件测试

包括5个适用于任何生命周期或机构的标准

ISO/IEC 29119-1：概念与定义

ISO/IEC 29119-2：测试过程

ISO/IEC 29119-3：测试文档

ISO/IEC 29119-4：测试技术

ISO/IEC 29119-5：关键词驱动测试

机载系统和设备认证中DO-178C软件的考虑事项

将测试需求与软件错误的后果联系起来：

A级：灾难性

B级：危害性/很严重

C级：严重

D级：轻微

E级：无影响

机载系统和设备认证中DO-178C软件的考虑事项

根据关键性等级，需达到以下测试覆盖：

DO-178C A级：

修订条件判定结构覆盖 (MC/DC)

判定/条件结构覆盖

语句结构覆盖



DO-178C B级：

判定/条件结构覆盖

语句结构覆盖



DO-178C C级：

语句结构覆盖

何时停止测试



| 时间/资金不足

| 未发现更多缺陷

| 证实了满足所有需求

| 证实了代码覆盖

| 达到可靠性目标

| 客户满意

ISTQB伦理规范



| 公众：

- 认证的软件测试人员的行为应符合公众利益

| 客户与雇主：

- 认证的软件测试人员的行为应符合其客户和雇主的最大利益，且与公众利益相符

ISTQB伦理规范



| 公众：

- 认证的软件测试人员的行为应符合公众利益

| 客户与雇主：

- 认证的软件测试人员的行为应符合其客户和雇主的最大利益，且与公众利益相符

| 产品：

- 认证的软件测试人员应确保他们提供的交付物（所测试的产品和系统）符合最高的专业标准

| 论断：

- 认证软件测试人员应保持其专业论断的正直性和独立性

ISTQB伦理规范



| 管理：

- 认证的软件测试经理和负责人应支持并促进软件测试管理的伦理规范

| 职业：

- 认证的软件测试人员应当维护其职业的正直与声誉，且符合公众利益

ISTQB伦理规范



| 管理：

- 认证的软件测试经理和负责人应支持并促进软件测试管理的伦理规范

| 职业：

- 认证的软件测试人员应当维护其职业的正直与声誉，且符合公众利益

| 同事：

- 认证的软件测试人员应当公平地对待同事，为其给予支持，并促进与软件开发人员的合作

| 本人：

- 认证的软件测试人员应参与有关其职业实践的终身学习，并应推动其专业实践相关的伦理规范

总结





基于规格说明的测试

- 第一部分

基于场景的测试

目标



目标

对比基于需求的测试
和基于场景的测试

功能测试



| 把系统作为黑盒
进行测试

| 以客户使用系统的方式
来测试系统

| 根据以下文档开发测试用例：

- 用户级文档
- 需求规格说明文档

用于开发测试的策略



| 静态（需求驱动）

- 传统方法
- 主要是一种验证活动

| 行为（场景测试）

- 既是验证活动，也是确认活动

需求驱动测试



| 仔细检查需求并开发测试用例，以确保每个需求都已通过一个或多个测试验证

| 通常一个测试将验证多个需求

| 演示需求对测试用例的可追踪性

需求可追踪性的工具



| 有许多现成的工具，都
 可让软件开发人员和工
 程师“共享”不断进化的
 需求

| 这些工具有助于将测试
 关联到需求，从而支持
 必要的可追踪性

需求可追踪性示例

| 针对 ATM 银行系统，考虑以下部分需求集：

1.0 系统应通过银行卡和 PIN 识别客户

1.1 系统应提示插入卡

1.2 系统应验证卡的信息

1.2.1 系统应没收被盗卡。

1.2.2 应退回无法识别的卡，并显示一条错误消息

需求可追踪性示例（续）



| 针对 ATM 银行系统，考虑以下部分需求集：

1.3 系统应提示输入 PIN

1.4 系统应验证输入的 PIN

1.4.1 如果 PIN 无效，系统应提示重新输入，最多 3 次。

1.4.2 如果超过 3 次输入都无效，系统应没收卡。

需求可追踪性示例（续）

| 针对 ATM 银行系统，考虑以下部分需求集：

2.0 系统应向客户展示交易选项

3.0 系统应处理交易请求

3.1 系统应支持取款交易

3.1.1 系统应提示输入金额和资金来源

需求可追踪性示例（续）

| 针对 ATM 银行系统，考虑以下部分需求集：

3.1.2 系统应验证余额是否足够

3.1.2.1 如果余额不足，系统应提示重新输入

3.1.3 系统应更新帐户并支出现金

4.0 在交易请求结束时，系统应生成收据并退回客户的卡

4.1 如果卡未取走，系统应没收卡

需求可追踪性示例

| (部分测试用例仅显示输入) 测试 #1

客户插入有效卡，输入的 PIN 正确，请求的取款金额适当，然后取走钱和卡。

覆盖的需求： 1.0、1.1、1.2、1.3、1.4、2.0、3.0、3.1、3.1.1、3.1.2、3.1.3、4.0

需求可追踪性示例



|测试 #2

客户插入被盗卡。

覆盖的需求：1.0、1.1、1.2、1.2.1

场景测试



|使用用例

- 所有功能需求都在使用用例中捕获（应该识别使用用例处理的需求）
- 许多非功能需求可视为使用用例的属性
- 使用用例提供一种方法来组织和抽象需求

使用用例测试方法



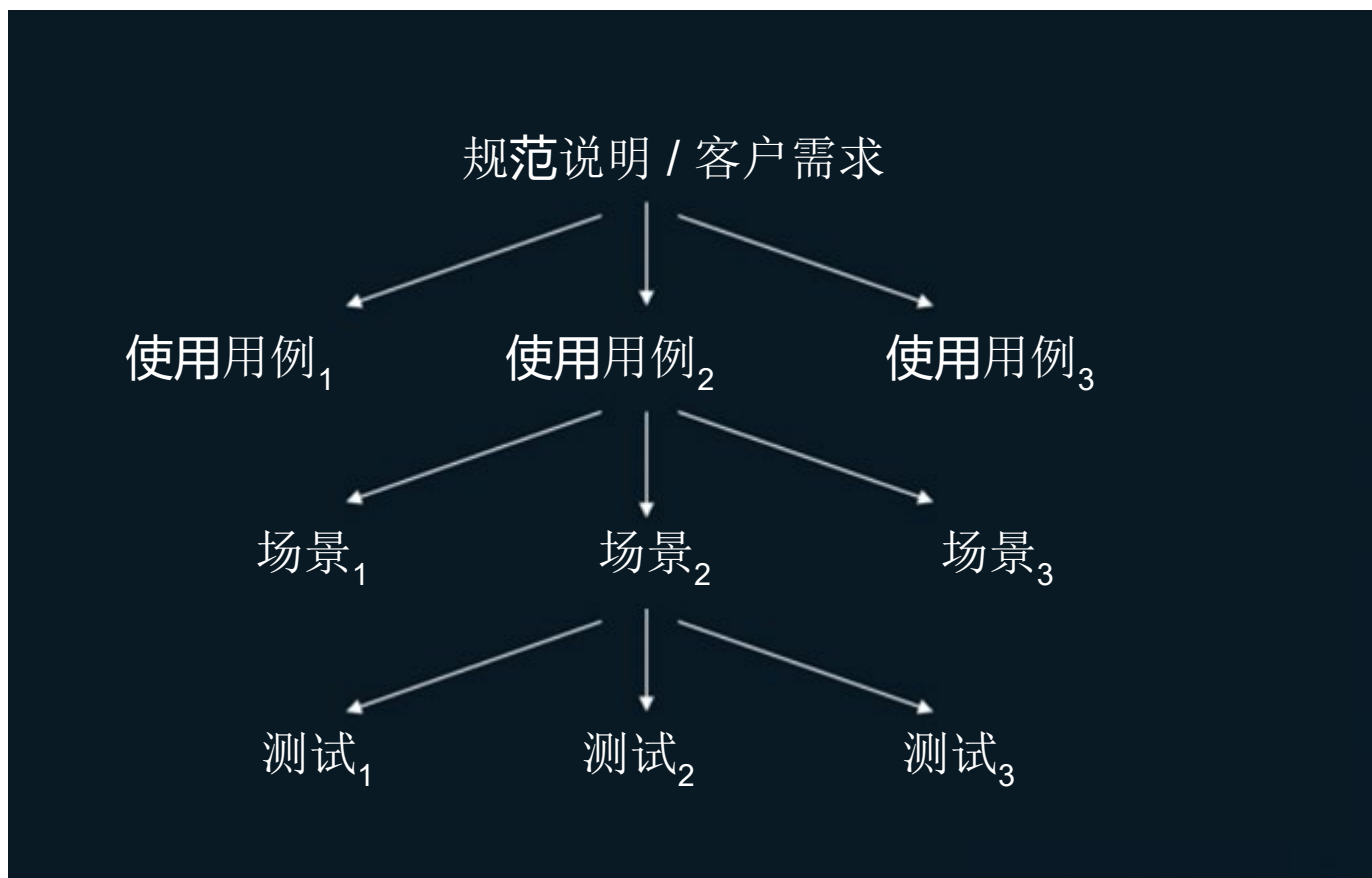
| 从规范说明中开发一组
使用用例（如果不是已
经存在）

| 验证和确认使用用例

| 针对使用用例开发场景

| 测试每个场景

场景测试



使用用例验证和确认



| 必须仔细检查使用用例
以确保它们完整且正确

| 确保所有客户需求都已
解决

| 确保行为者之间的所有交互都正确

| 确保所有需求都已覆盖

使用用例构建



| 确定行为者（确定系统范围）

| 针对每个行为者，确定行为者如何使用系统完成功能

| 详细描述各使用用例，将每个处理流程确认为一个场景

建立使用用例文档



| 可以将使用用例记如文档：

1. 分步说明
2. 状态图

示例（分步说明）

考虑 ATM 系统和用例“取款”



分步说明示例（续）



| 使用用例中的每个路径都代表一个必须测试的场景。

分步说明示例（续）

| 备选路径（可能性子集）：

- 3a. 如果是被盗卡，将吞卡。
- 3b. 如果无法识别帐户，将退卡并生成消息。
- 5a. 如果 PIN 不正确，提示重试。5b.
如果 3 次尝试都无效，将吞卡。
- 7. 如果余额不足，将生成消息并允许重新输入。
- 13. 如果未取走卡，将吞卡。

总结





基于规格说明的测试

- 第一部分

等价类划分

目标



目标

应用等价类划分

测试技术

等价类划分



| 将程序/功能的输入域划分为有限数量的等价类的技术

- 同时考虑有效和无效的等价类

| 从每个等价类中选择一个或多个输入

| 通过将等价类映射到测试用例来演示功能覆盖

| 等价类划分适用于测试输入相互独立的功能

等价类划分步骤



1. 对于每个输入，识别一组等价类并进行标记
2. 编写测试用例，尽可能覆盖尚未覆盖的有效等价类
3. 对于每个无效等价类，编写覆盖其中一个尚未覆盖等价类的测试用例

等价类划分示例

测试如下过程：

- Validate_New_Password,
此过程将接受用户输入的密码，并验证新密码是否符合以下规则：

1. 密码长度必须介于 6-10 个字符之间
2. 第一个和最后一个字符必须是字母、数字或“?”
3. 剩余字符可以是除控制字符以外的任何字符
4. 密码不得使用字典单词

等价类分区

长度	第一个字符	最后一个字符	其余字符	密码状态
(1) 6-10	(2) 字母	(5) 字母	(8) 常规	(17) 字典单词
(10) < 6	(3) 数字	(6) 数字	(16) 控制字符	(9) 非字典单词
(11) > 10	(4) ?	(7) ?		
	(12) 控制字符	(14) 控制字符		
	(13) 其他	(15) 其他		

测试

$\frac{X}{1..10}$ $\frac{Y}{50..100}$ $f(X,Y) \Rightarrow \begin{matrix} \text{Result} \\ \text{Error} \end{matrix}$

$\frac{X}{1..10}$	1	2	3	4	5	6
1..10 (V)	✓	✓	✓			
< 1 (F)				I		
> 10 (F)					I	

$\frac{Y}{50..100}$	1	2	3	4	5	6
50..100	✓			✓	✓	
< 50		I				
> 100			I			

X	Y
5	55
6	45
7	105
-1	40 70
15	50

测试矩阵



EP

总结





基于规格说明的测试

- 第一部分

边界值测试

目标



目标

应用边界值

测试技术

边界值测试



|选择测试用例，以测试等于、大于和小于输入与输出等价类划分边界值的情况下所对应的边界条件

边界值测试



总结





基于规格说明的测试

- 第一部分

因果分析

目标



目标

应用因果

测试技术

因果分析



|在测试功能时是必要的，

如果必须同时测试输入

组合

|利用决策表和决策树

示例

考虑一个功能拥有两个输入变量：客户和订单，

一个输出变量：折扣。

客户可以是类型 A、B 或 C，订单的范围是 1 至 1000。该功能是根据客户类型和订单计算折扣。在这个示例中，假设应用以下规则。

1. 客户类型 A：若购买数量小于 10，折扣为 0%；若购买数量为 10 到 99，折扣为 5%；若购买的商品为 100 件或以上，则折扣为 10%。
2. 客户类型 B：若购买数量小于 10，折扣为 5%；若购买数量为 10 到 99，折扣为 15%；若购买的商品为 100 件或以上，则折扣为 25%。
3. 客户类型 C：若购买数量小于 10，折扣为 0%；若购买数量为 10 到 99，折扣为 20%；若购买的商品为 100 件或以上，则折扣为 25%。

因果分析 / 决策树



总结





基于规格说明的测试

- 第一部分

测试异步事件

目标



目标

测试异步事件

外部事件



| 在现实世界中发生、而对
系统有重要影响的事件

| 通常描述为行为者与系统
之间的消息

时间线



| 用于对异步事件建模

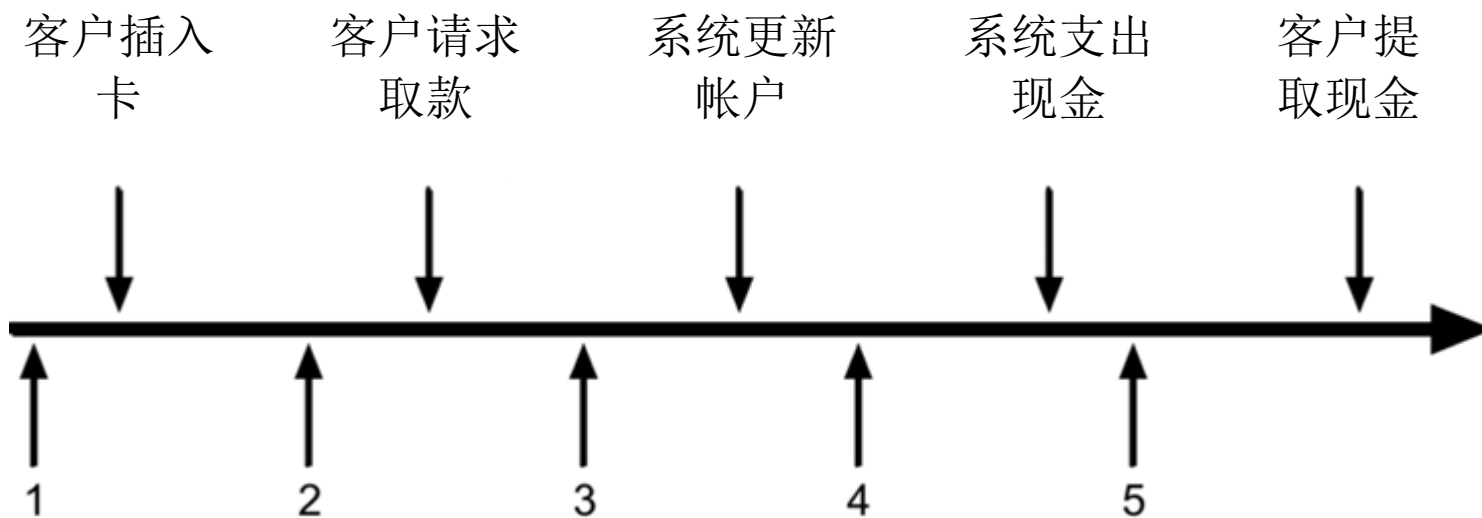
| 第一步包括识别重要的
使用用例活动，并将它
们放在从左到右的时间
线上

| 可以根据沿时间线发生的
异步事件生成测试

ATM 示例 - 时间线

重要的使用用例活动可以放在时间线上，以评估异步事件的影响

可以根据沿时间线发生的事件生成测试



总结



基于规范说明的测试

- 第一部分

基于状态的测试

目标



目标

应用基于状态的
测试技术

状态机



| 有时采用状态机的形式
最能捕获系统行为

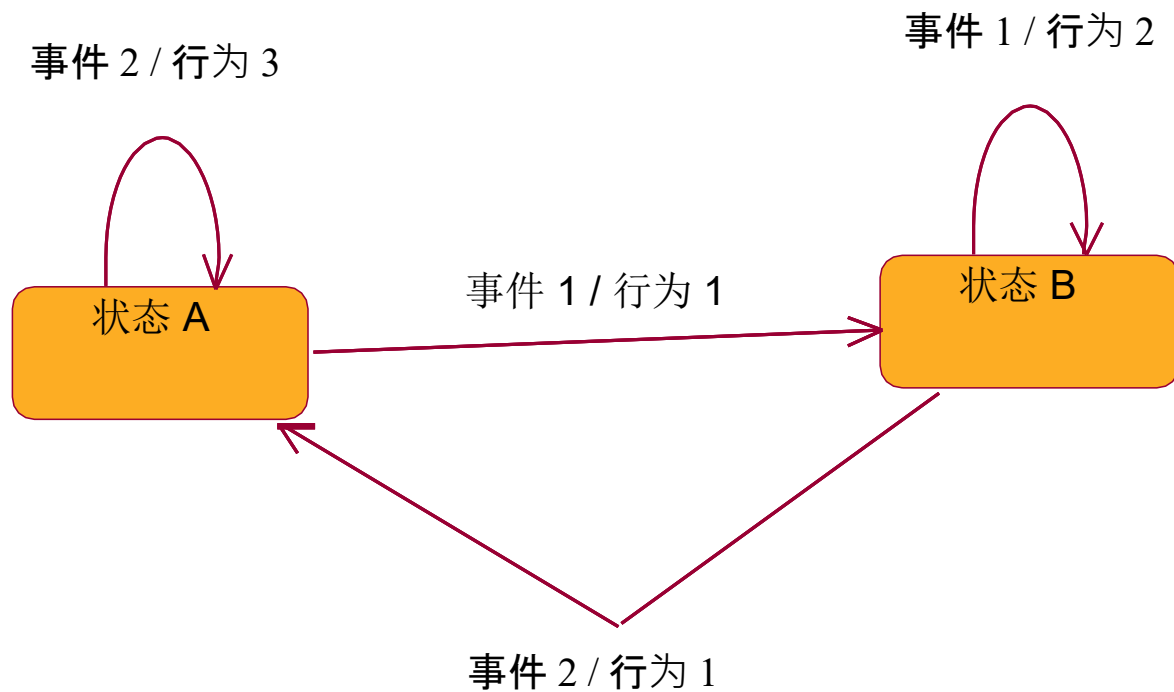
| 状态机包括一组状态
和事件

| 当在特定状态中发生某
个事件时，它将触发状
态转换和响应

状态机的表格表示

	状态 A	状态 B
事件 1	状态 B / 行为 1	状态 B / 行为 2
事件 2	状态 A / 行为 3	状态 A / 行为 1

状态机图示

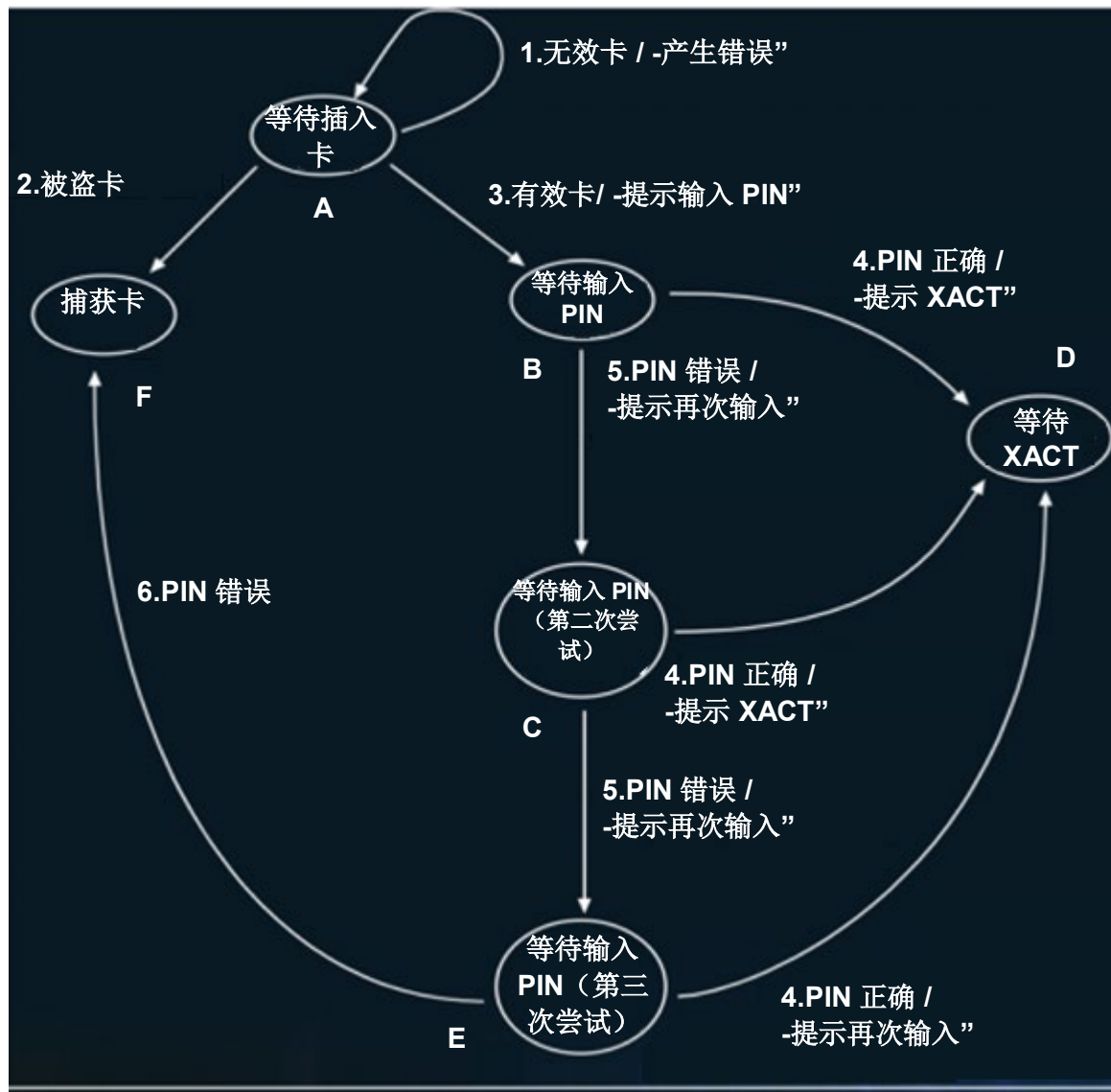


状态图示例



| 考虑有关在 ATM 系统中确认用户身份的
使用用例

状态机示例



基于状态的测试



必须检查状态机的：

|完整性

- 必须识别每个状态/事件对
- 验证条件转移是否正确

|矛盾

- 来自同一状态的 2 个转移不应包含相同的事件
- 嵌套的状态图可能有危险

基于状态的测试



必须检查状态机的：

| 无法到达的状态

- 具有嵌套并使用条件转移的状态机可能很复杂

| 死锁状态

- 可以进入但无法退出的状态

测试有限状态图



通过以下步骤，可为有限状态图开发测试覆盖：

1. 开发一个状态测试树
2. 识别测试序列（历经此树的路径）
3. 开发测试以包含这样的序列，从初始状态开始，并以可观察的行为结束

开发状态测试树



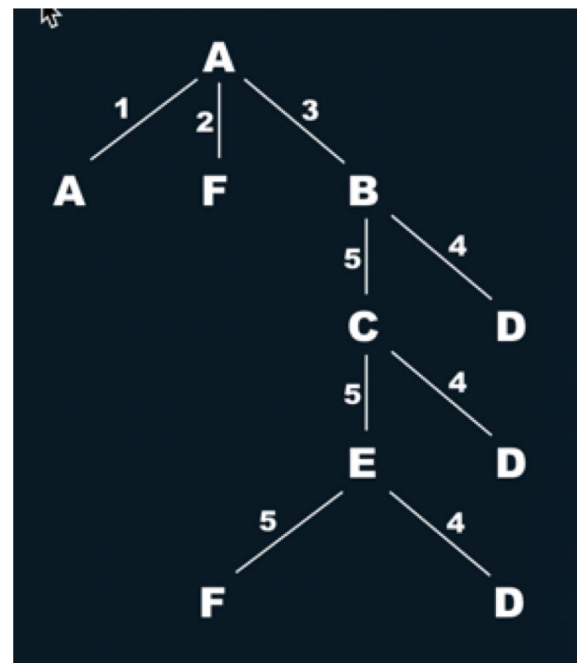
| 从初始状态开始，作为树的根

| 在第一个层次上，识别来自初始状态的每个转移和到达的新状态

| 继续向下扩展此树，针对在更高层次上，在之前没有在树中出现的每个状态

状态测试树

测试序列	测试
1	1
2	2
3, 4	3, 4
3, 5, 4	3, 5, 4
3, 5, 5	3, 5, 5
3, 5, 5, 4	3, 5, 5, 4



测试将完全映射到测试序列，因为
每个序列都将生成可观察的行为。

总结





基于规格说明的测试

- 第一部分

基于模型的测试

目标



目标

了解基于模型的
测试策略

基于模型的软件开发



| 针对系统的行为开发

一个可执行的模型

- 例如 UML

| 模型可以进行分析和模拟

| 利用工具从模型中生成代码

基于模型的测试 (MBT)

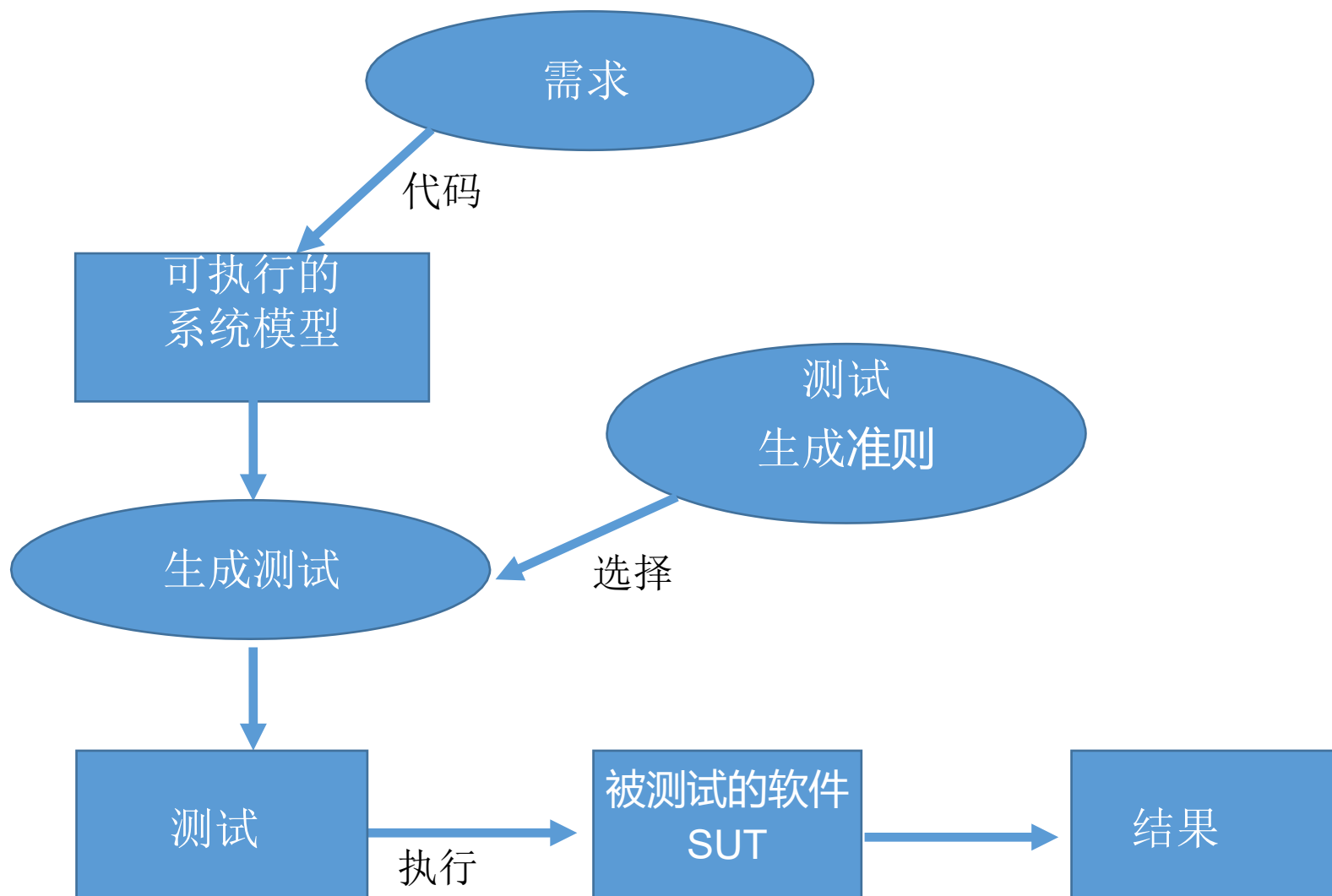


| 根据系统模型生成测试

| MBT 步骤

- 创建系统模型
- 选择**一些**测试生成标准
(例如 EP 或 BV)
- 生成测试
- 执行测试

MBT 过程



MBT 的主要优点



| 建模可提供精确性并减少歧义

- 有助于程序验证

| 为生成自动化测试提供了潜在可能性

| 行为变更可直接转换为测试变更

- 对于具有易变需求的程序尤其有价值

总结





基于规格说明的测试

- 第二部分

有关测试质量的

组合覆盖

目标



目标

应用组合测试覆盖来评估测试质量

评估测试质量

有许多方法可以评估测试质量，包括：

- 组合覆盖
- 变异测试

组合覆盖着眼于如何一起测试参数值的组合

多项研究表明，通过少量参数的组合可以检测到大多数故障

https://ws680.nist.gov/publication/get_pdf.cfm?pub_id=917352

t 路总覆盖率

| 对于包含“n”个变量和值的
给定测试集，执行的 t
路变量值组合的比例

| 例如假设我们在测试一
个有 3 个变量的函数：

- 变量 a：值为 0 和 1
- 变量 b：值为 0 和 1
- 变量 c：值为 0 和 1

| 通过以下测试实现的 2 路
变量值配置的总覆盖率是
多少？

a=0; b=0; c=0

a=1; b=1; c=1

a=1; b=0; c=0

总结





基于规格说明的测试

- 第二部分

实验设计

目标



目标

应用实验设计来
开发测试

背景



| 实验设计 (DOE) 是用于
评估系统或过程的系统
化方法

| DOE 大量应用于制造和
质量工程方面

| DOE 能够有效地检查系
统的行为

传统实验



| 传统的系统行为评估包括设计一个实验，在此实验中修改一个因素，然后评估系统的行为

| 例如，考虑改变烤箱的温度对披萨口感的影响

传统方法的缺点



| 大多数系统的行为受许多因素影
响

| 各种因素组合也可能相互作用

| 在披萨用例中，其他因素包括：

- 烤架位置
- 烤制时间

DOE 优点



| DOE 能够检查单个因素以及因素组合的影响

| 必须确定每个要检查的因素的值/范围

| 结合要考虑的因素及其对系统的影响进行实验（运行）

披萨示例因素



| 烤制时间

- 低 / 中 / 高

| 烤架位置

- 1 / 2 / 3 / 4 / 5

| 温度

- 350 / 375 / 400

全面的 DOE 组合



| 运行 45 次

DOE 分类



|全面因素设计

- 测试**每种**因素值组合
- 披萨示例

|部分因素设计

- 仅处理所有组合中的一部分
- **正交数组法**测试通常用于处理有限的因素组合

实验设计结对组合



1. 识别定义每个配置的参数
2. 对每个参数进行分区
3. 指定用于禁止配置分区的特定组合的约束

实验设计结对组合

4. 指定要测试的配置，覆盖所有能满足约束的配置

参数分区结对组合

- “对于任何两个参数 $P1$ 和 $P2$ ，以及任何用于 $P1$ 的分区值 $V1$ 和用于 $P2$ 的分区值 $V2$ ，有一个指定的配置，即 $P1$ 具有值 $V1$ ， $P2$ 具有值 $V2$ 。”

披萨示例

1	中	350
---	---	-----

2	低	350
---	---	-----

3	高	350
---	---	-----

4	低	350
---	---	-----

5	低	350
---	---	-----

1	低	375
---	---	-----

2	高	375
---	---	-----

3	中	375
---	---	-----

4	中	375
---	---	-----

5	中	375
---	---	-----

1	高	400
---	---	-----

2	中	400
---	---	-----

3	低	400
---	---	-----

4	高	400
---	---	-----

5	高	400
---	---	-----

DOE 在软件测试方面的经验



| 多家公司在软件测试中
使用了 DOE，并报告
了良好的结果

| 已经证明 DOE 能够达到合
理的代码覆盖率

警告



|许多软件功能包含许多
参数和因素

|结对组合测试可能会使许
多在正常的日常场景中的
功能未经测试

总结





基于规格说明的测试

- 第二部分

变异测试

目标



目标

了解蜕变测试

测试预言难题



| 一个机构开发了一组测试

| 该机构针对程序执行测试

| 所有测试都通过

| 我们可以得出什么结论？

变异测试



| 将缺陷（变异体）引入
到正在测试的程序中

| 检查测试用例是否可以
检测到变异体

| 这项工作开始于 70 年代早期，但由于成本问题而没有被广泛采用

| 如今的自动化测试环境
使变异测试变得可行

创建变异体



| 变异体通常是通过修改
源代码的语法创建的

| 变异生成工具就是为了
执行此工作

| 变异示例

- 修改布尔表达式 ($<$ vs $\leq$)
- 删除语句
- 修改变量
- 修改算数运算

变异测试假设



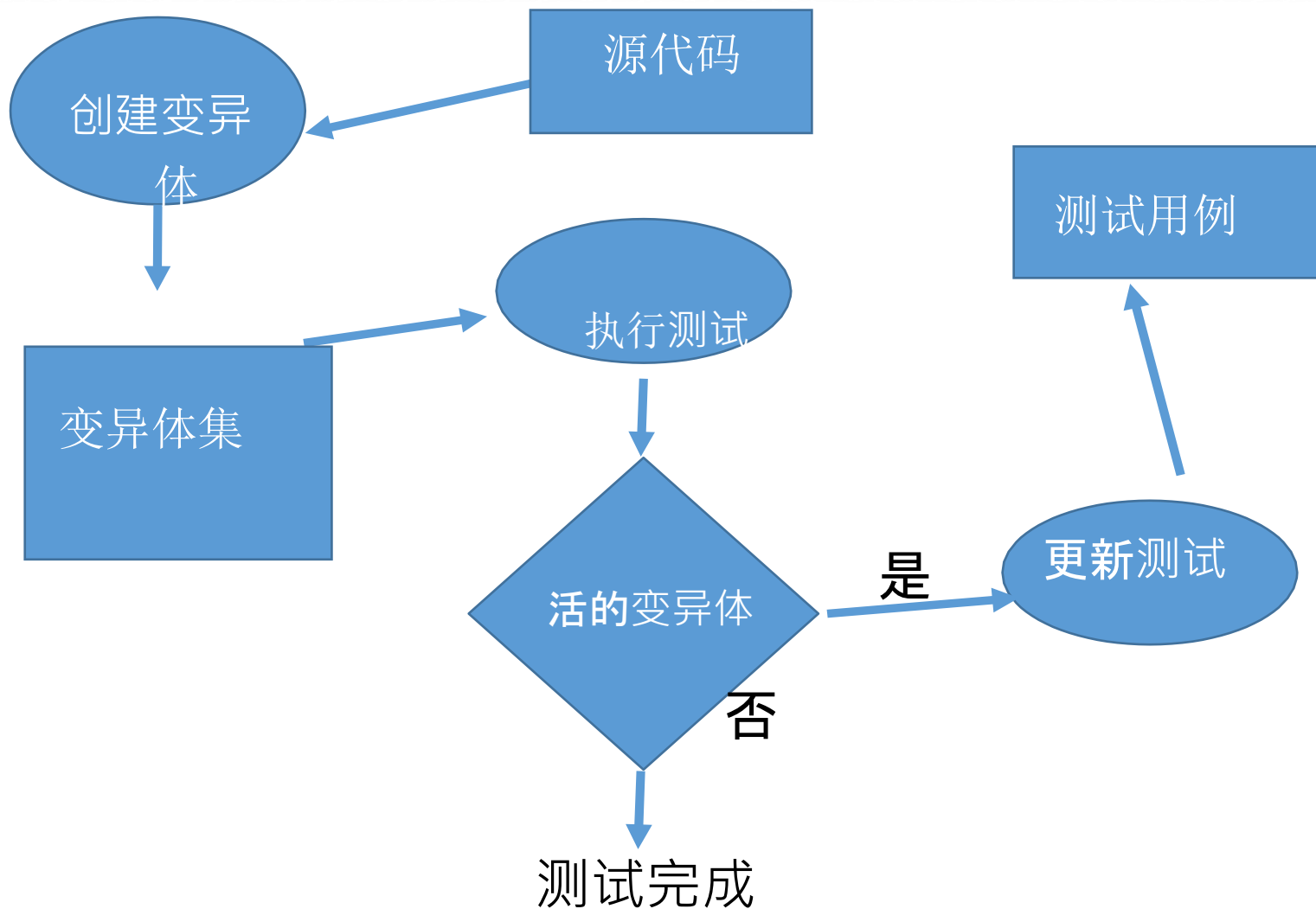
| 程序员的能力假设

- 程序员通常创建接近正确的代码，只有微小的错误

| 耦合效应

- 这是一种信念，相信可以检测到小错误的测试数据也可以检测到复杂的错误

变异测试过程



总结





基于规格说明的测试

- 第二部分

模糊测试

目标



目标

了解模糊测试

模糊测试



| 用于测试何处将自动生成无效、随机或非预期的输入的方法

| 黑客通常藉此发现系统的漏洞

| 不需要测试预言

- 仅监视崩溃或不良行为

| 用于生成输入的模糊测试工具

两种类型的测试生成器



| 基于变异

| 基于生成

基于变异的模糊测试

通过对有效的测试数据进行随机修改来生成测试输入

不需要有关输入的知识

修改可能是完全随机的，也可能遵循一些与如下常见的错误类型相关的模式：

- 长字符串或空白字符串
- 最大值或最小值
- 特殊字符

一些工具使用“位翻转”- 通过更改输入中的随机位来损坏输入

基于生成的模糊测试



| 根据测试输入格式的
规格说明生成随机的
测试数据

| 在输入中的每个可能位
置添加异常值

| 协议的知识应该比随机的
模糊测试提供更好的
结果

何时停止模糊测试？



| 利用代码覆盖工具

总结





基于规格说明的测试

- 第二部分

蜕变测试

目标



目标

了解蜕变测试

测试预言难题



| 对于许多应用程序，很难确定预期的结果

- 图形应用程序
- 复杂处理
- 机器学习
- 大数据

| 模糊测试可用于检测崩溃

| 蜕变测试可以提供帮助

假设



| 某些程序具有这样的属性，当对输入进行更改时，可以预测输出的改变（蜕变属性）

| 考虑一种计算数字序列方差的函数

- 更改顺序有什么关系？
- 添加 10 来递增数字有什么关系？

蜕变测试

初始测试
用例

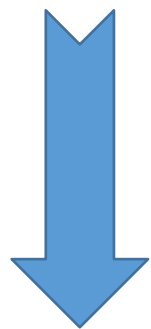
X



$f(X)$

函数

根据蜕变属
性更改



$P(X)$



$f(P(X))$

函数

根据蜕变关
系检查

标准偏差示例



| 初始测试： 10、20、30、40、50

– 结果： 14.142

| 第二个测试： 20、30、40、50、60

– 结果： 14.142

| 第三个测试： 10、30、50、70、90

– 结果： 28.284

总结





基于规格说明的测试

- 第二部分

基于缺陷的测试

目标



目标

应用基于缺陷
的测试技术

基于缺陷的测试



| 利用缺陷分类法生成测试用例

| 分类法是一个系统，用于对层次化的类别进行分类

| 缺陷分类法提供了对软件缺陷的分类

| 现时有多种缺陷分类法

| 通常从过去检测到的缺陷开发和演变而来

Beizer 通用缺陷分类法类别



|需求缺陷

|功能缺陷

|结构缺陷

|数据缺陷

|实现和编码缺陷

|集成缺陷

|系统和软件体系结构缺陷

|测试定义和执行缺陷

|未分类的缺陷

社区开发的软件缺陷列表



<http://cwe.mitre.org/data/definitions/699.html>

基于缺陷的测试方法

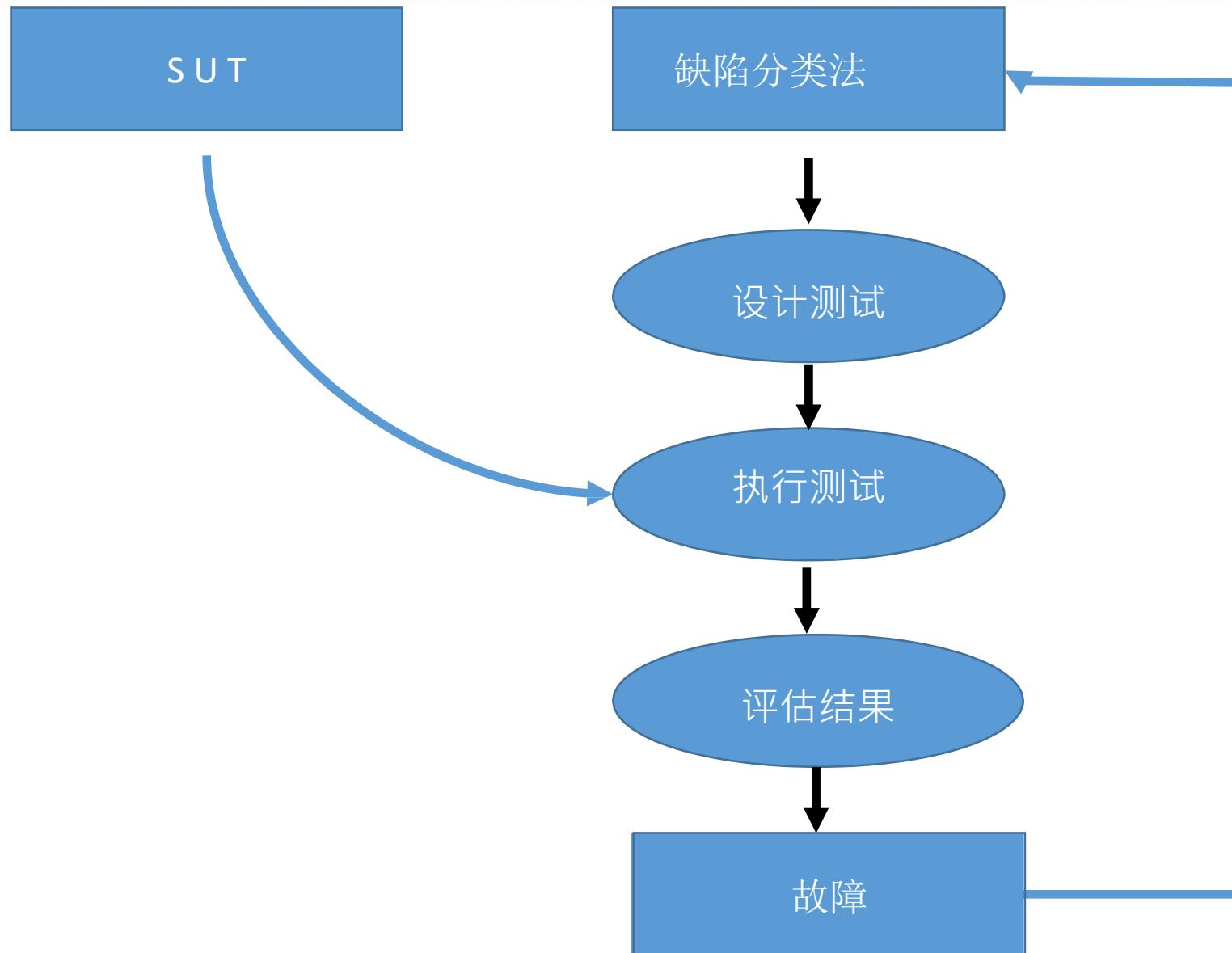


| 开发针对特定缺陷类别
的测试用例

| 可应用于任何级别的测试

| 示例： 考虑开发
针对计算中除零错误的
测试

基于缺陷的测试过程



总结





基于规格说明的测试

- 第二部分

探索性测试

目标



目标

了解探索性测试

与早期探险者类比



| 在探索之前尽量了解情况

| 开发用于探索的系统化策略

| 持续跟踪你的探索过程

| 注意可能的副作用

| 仔细记录发现

探索性测试



| 与脚本测试不同，测试人员将在探索产品的同时编写测试用例

| 测试由需求和之前的测试结果驱动（持续学习）

| 有可能检测到脚本测试和自动化测试遗漏的错误

探索性测试（基于测程的测试）

| 一对测试人员在 90 分钟
的测程内协同工作

| 测试侧重于主题/浏览（测试内容）

- 类似于在城市中游览
- 令探索系统更有结构（应用程序浏览、**功能**浏览、**菜单**浏览），同时侧重于你正在查找的不同类型的错误

| 将生成测程报告

- 测试了什么内容
- 结果
- 错误

浏览示例

需求浏览：

- 查找软件中告诉用户产品或某个功能的预期操作的所有信息。是否能提供充分解释？
结果是否反映了提出的主张？

复杂性浏览：

- 查找最复杂的功能和数据，换言之，就是最难解决的错误可能隐藏的所有位置

持续使用浏览：

- 使系统长时间保持打开状态，并且打开多个屏幕和文件。观察随着硬盘和内存使用量增加会发生什么

文档浏览：

- 浏览产品的帮助部分，并遵循一些说明查看它们是否能产生预期的结果

浏览示例



| 功能浏览：

- 尽可能多地尝试应用程序上可用的控件和功能

| 互操作性浏览：

- 检查系统是否按预期与第三方应用交互，以及数据是否按预期进行共享和更新

| 场景浏览：

- 创建一个场景（用户案例）来模仿用户与系统的真实交互的，并呈现该场景

| 可变性浏览：

- 查找系统中可以更改或自定义的所有元素并测试设置的不同组合

总结





基于结构的测试策略

控制流测试

目标



目标

开发测试用例，
以实现控制流覆
盖

代码覆盖



| 分析通过执行基于需求的测试用例所获得的代码覆盖率是很重要的。

| 对代码覆盖率进行评估的方法包括：

- 控制流
- 数据流

| 无法获得覆盖率的原因可能包括：

- 代码中包含未记录的需求
- 死代码
- 需求的测试用例不完整

控制流覆盖等级



| 语句覆盖

| 判定覆盖

| 判定/条件覆盖

| 多重条件覆盖

语句覆盖

|开发测试用例，让每条语句至少执行一次。

```
if a < 10 or b > 5
```

```
then
```

```
    x := 50
```

```
else
```

```
    x := 0 ;
```

```
    if w = 5 or y > 0
```

```
    then
```

```
        z := 48
```

```
else
```

```
    z := 5 ;
```


决策(判定)覆盖



开发测试用例，让每条分支至少走过一次。

| 分支语句的示例是什么样的？

| 语句覆盖能满足决策覆盖吗？

| 决策覆盖能满足语句覆盖吗？

决策/条件覆盖

| 开发测试用例，使决策的每个条件至少把所有可能的结果执行一次，并且每个决策至少把所有可能的结果执行一次

多重条件覆盖



| 开发测试用例，对判定中多个条件的所有组合进行测试

二分检索示例

inputs: table, num, key

outputs: found, loc

start := 1;

end := num;

found := false

while start <= end and not found

 middle := (start + end) / 2

 if key > table [middle]

 then start := middle + 1

 else if key = table [middle]

 then found := true

 loc := middle

 else end := middle - 1

总结





基于结构的测试策略

结构测试

目标



目标

应用结构测试技
术

McCable 圈型复杂度

| 具有 e 个边、 n 个节点和
 p 个连通分量的图的 $v(G)$
为 $e - n + p$

| 在典型程序中：

$$-v(G) = \text{测试谓词数量} + 1$$

示例

```
S1;  
if x < 10  
    then S2  
    else if y > 0  
        then S3  
        else S4;  
If z = 5  
    then S6  
    else S7;
```

测试应用



| 无法测试所有通过代码
的路径

| 结构测试提供了一种测试
路径子集的方法

| 选择一组基本路径集（数字
为 $v(G)$ ）

| 基本路径的线性组合将
生成任何路径

| 保证分支覆盖率

确认基本路径



| 选择一个通过“图”的任意
路径，作为初始基础路
径

| 保持其他判定不变的情况
下，翻转第一项判定

| 重置第一项判定，并翻转
第二项判定

| 继续，直至所有判定均
被翻转

示例



S1;

if $x < 10$

then S2

else if $y > 0$

then S3

else S4;

If $z = 5$

then S6

else S7;

总结





基于结构的测试策略

数据流测试

目标



目标

开发测试用例以
实现数据流覆盖

方法



| 使用 3 组变量为每个节点注释控制流图

| Def(i) - 在节点 I 中定义的一组变量

| C-Use(i) - 在节点 I 的计算中使用的一组变量

| P-Use(i) - 在测试谓词中使用的一组变量

示例



Get x,z;

y := 0;

If x > 10

 then y := 15;

If z > 0

 then w := y+1

 else w := y-1;

定义显然路径



| 对于变量 x ，从节点“ i ”到节点“ j ”的定义显然路径如下： x 在节点 j 中定义，并在节点 j 的测试谓词或计算中使用， x 在节点 i 和节点 j 之间没有被重新定义

示例

```
get x,y;  
a := 0;  
b := 0;  
if x > 10  
    then w := a+1  
         b := 4  
    else w := b+1  
         a := 4;  
If y > 10  
    then z := a+w  
    else z := b+w;
```

定义使用（DU 路径）覆盖



| 针对变量的每个定义，
开发测试用例以执行所
有 DU 路径

| DU 路径以变量的定义开
始，并以定义显然路径中
变量的计算使用或谓词使
用结束

示例

```
get x,y;  
a := 0;  
b := 0;  
if x > 10  
    then w := a+1  
         b := 4  
    else w := b+1  
         a := 4;  
If y > 10  
    then z := a+w  
    else z := b+w;
```

总结





基于结构的测试策略

静态分析

目标



目标

确定静态分
析技术

数据流分析



| 对程序中的数据流进行
建模

- 变量在哪里定义
- 变量在哪里使用

| 在不执行程序的情况下
执行分析

| 查找数据流异常

数据流异常示例



| 定义并重新定义了变量，
但未引用

| 引用未定义的变量

| 定义变量，但从不使用
它

| 有许多工具可用于执
行异常检测

黄氏定理



设 A 、 B 、 C 为字符序列的非空集，其最小字符串长度至少为 1 个字符。

设 T 为 2 个字符的字符串。如果 T 是 Ab^nC 的子字符串， T 将出现在 AB^2C 中。

总结





基于结构的测试策略

符号执行

目标



目标

利用符号执行

符号执行



| 用于形式化描述路径域特征和路径条件的技术

| 程序中的所有路径形成一个执行树

| 涉及使用符号值执行程序

| 确定测试数据以执行一条路径或确定一条路径是否不可行

标记



| 在遍历路径时，变量
“x”将具有一系列符号
值： A_0 、 A_1 、 A_2

| 下标是指前一个执行的语
句的编号

示例



- (0) input A,B
- (1) $A := A + B;$
- (2) $B := A + B;$
- (3) $A := 2 \times A + B;$
- (4) $C := A + 4;$

多个路径示例

```
if (x <= 0) or (y <= 0) then
```

```
(1)   x := x2;
```

```
      y := y2;
```

```
      else
```

```
(2)   x := x + 1
```

```
      y := y + 1
```

```
      endif
```

```
      if (x < 1) or (y < 1) then
```

```
(3)   x := x + 1;
```

```
      y := y + 1;
```

```
      else
```

```
(4)   x := x - 1;
```

```
      y := y - 1;
```

```
      endif
```

示例



路径条件



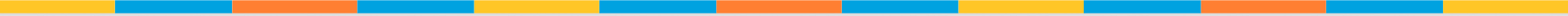
|除了符号化地评估路径中的程序变量，我们还可以符号化地表示遍历该路径所需的条件

|符号路径条件必须用变量的初始符号值表示

T,F 路径示例



T,F 路径示例



总结





测试软件质量特性 – 第 1 部 分 性能测试

目标



目标

利用性能测试的
策略

性能测试



| 目标

| 验证系统是否满足指定负载条件（包括压力和容量情况）下的性能要求。

性能测试入口准则



| 量化和可测量的
性能需求

| 适度稳定的系统

| 测试典型的客户端
环境

| 工具

- 负载生成器
- 资源监控器

测试数据管理



| 就当今的应用程序而言，
数据日益复杂，数据量
日益庞大，需要注重测
试数据管理

| 从操作系统复制真实数据
的完整数据集是理想之选

练习



| 假设你是航空公司新预订系统的系统测试员。

| 讨论如何改善以下性能要求：

–“航空公司预订系统应具有出色的响应时间。”

负载规格说明

| 对于相关**使用**用例，就
负载说明性能需求非常
重要

| 负载可能反映：

- 不同的活动量
 - 手机系统的忙时
 - 使用相同的**通话轮廓**
- 不同的活动组合
 - 手机区域的地震或龙卷风情况
 - 不同的**通话轮廓**

性能测试期间不断变化的负载



| 对于相关**使用**用例，应
改变负载并跟踪响应时
间

- 验证性能需求

| 随着负载变化，还可以跟
踪资源使用情况，以便确
定潜在的瓶颈或性能问题
来源

总结





测试软件质量特性 – 第 1 部 分 压力测试

目标



目标

利用压力测试的
策略

压力测试



| 在资源饱和并超出其极限时，验证系统的行为是否满足其需求。

| 尝试找到压力点，并确保系统按规格说明运行

类比



| 人体压力测试

| 桥梁压力测试

| 汽车压力测试

压力测试步骤

1. 确定压力点

- 与开发人员和架构师合作
- 确定潜在瓶颈

2. 制定策略，以便在步骤

1 中确定的压力点上向

系统施加压力

- 通常需要负载生成工具

3. 验证确实产生了预期压力

- 压力测试策略可能无效
- 系统性能可能比预期好

4. 观察行为

- 验证是否满足压力相关要求
- 确保功能正确性

总结





测试软件质量特性 – 第 1 部分

容量测试

目标



目标

利用容量测试的
策略

容量测试



| 在系统长期承受大量活动时，验证系统行为是否满足其需求。

容量测试针对的错误



| 内存泄漏

| 资源耗竭

| 计数器溢出

总结





测试软件质量特性 - 第 1 部分

配置测试



目标



目标

利用配置测试的
策略

多重配置



| 是否应对每种可能的配置重复进行性能测试？

配置测试



验证对于系统运行所采用的不同配置，系统的功能和性能要求得到满足。

配置测试步骤

1. 确定定义可对系统满足功能和性能要求的能力产生影响的每种配置的参数

- CPU
- 操作系统
- 存储
- 数据库

2. 分区（将类似的参数分为一组，以减少可能的配置数量）

3. 识别将要测试的配置组合

- 边界值（最大和最小）
- 基于风险
- 试验设计结对组合（用于在测试所有组合不切合实际或不需
要之时选择配置参数组合）

示例：车的性能测试

<u>发动机</u>	<u>变速器</u>		<u>轮胎</u>
3.0	自动 (a)	2D	15 英寸普通 (15 n)
3.8	手动 (m)	4D	15 英寸高性能 (15 hp)
5.0			

配置组合有 24 种。

车的性能测试（续）



最慢的配置

3.0 / 自动 / 4D / 15 n

最快的配置

5.0 / 手动 / 2D / 15 hp

基于风险的选择依据预计的销售而定

3.8 / 自动 / 4D / 15 n

结对组合

<u>发动机</u>	<u>变速器</u>	<u>2D/4D</u>	<u>轮胎</u>
3.0	a	2D	15 n
3.0	m	4D	15 hp
3.8	a	2D	15 hp
3.8	m	4D	15 n
5.0	a	4D	15 hp
5.0	m	2D	15 n

总结





测试软件质量特性 – 第 1 部分

回归测试

目标



目标

利用各种回归测
试策略

回归测试

new

defects/KLOC

.1/KLOC

| 修改现有软件是一项高风险活动

| 进行修改的原因是：

- 错误修复 ✓
- 新功能整合 ✓

| 修改可能由于以下原因引入错误：

- 代码波动效应
- ~~非预期特征交互~~
- 性能、同步、~~资源共享等~~方面的变化

*3 fixes
1 → 1 new error*

示例

— R1 — "R2" system test

3.2 → 6.0

GTE/亚利桑那州立大学的研究

发现，对于在大型交换系统的系统测试期间检测到的问题，几乎有一半是先前版本中运行正常的功能

Caper Jones 的研究发现，不良修复注入率范围在低于 2% 到超过 20% 之间

阿丽亚娜 5 号运载火箭控制器

- 3.5 亿美元损失
- 没有回归测试代码
- 假设火箭的动力学与阿丽亚娜 4 号运载火箭相同

Base A2

目标

回归测试的目标在于确保在修改软件之后，先前开发和测试的功能可以继续按规格说明运行

回归测试必须从多层次角度进行

- 单元级回归测试
- 集成级回归测试
- 系统级回归测试

增量开发和测试过程概述

开发人员

系统测试

~~在构建 1 中实现第一组函数~~

~~测试 构建 1~~

~~创建回归测试~~

~~报告问题~~

将修复整合到新构建中

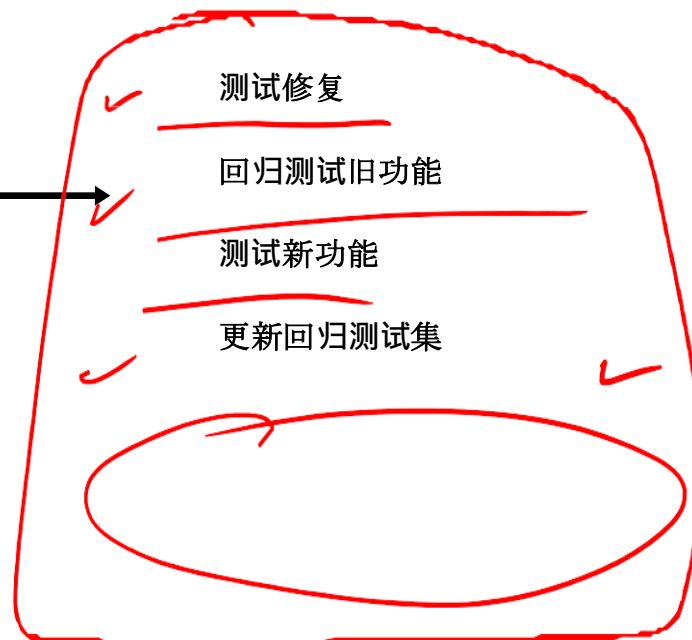
在新构建中实现下一组功能

✓ 测试修复

✓ 回归测试旧功能

✓ 测试新功能

✓ 更新回归测试集



详细的缺陷修复过程

系统测试

执行测试和检测错误

编写问题报告。

问题分析师

分析问题并建议纠正措施

变更控制委员会

决定修复、取消或推迟

开发修复代码

审查变更。

单元测试变更。

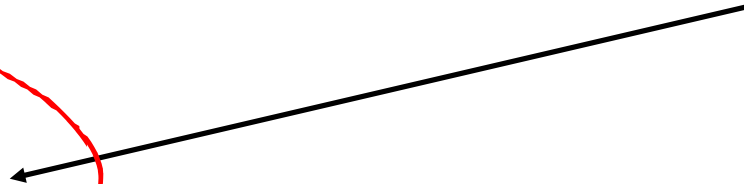
回归测试

已修改的组件。

添加变更到下一个构建。

测试修复。

回归测试。



回归测试策略

"Black Box"

✓ TURN Radio on
- Repeat the test that failed

全部

- 重新运行所有现有测试以响应代码修改
- 通常不切实际

used car
"AS-IS"

"3 minutes"

选择性

- 根据修改重新运行选定的测试子集
- 不管修改如何，执行标准置信测试

"20 minutes"

19' 55"

25"

基于修改的选择性回归测试

| 通常需要工具，并与开发
人员密切沟通

| 策略包括：

- ~~Delta~~ 代码测试 ✓
- 波动效应分析 ✓

Delta 代码测试

| 需要覆盖工具，用于将测试用例映射到所需粒度级别的代码：

- 代码块 ✓
- 组件 ✓

△ SCM

| 需要配置管理工具识别代码更改 **Delta**

| 策略建议重新运行遍历已更改或已删除代码的测试

示例

测试

coverage

↓

↓

↓

	T1	T2	T3	T4	T5
A	X	X	X	X	X
B	X		X		
C		X			X
D	X			X	
E			X		
F		X			X

如果更改了组件 B 和 D，则将重新执行测试 T1、T3 和 T4。

波动效应分析

| 要求开发人员确定更改对其他需求或特性的影响

| 可能受影响的需求和特性会传达给系统测试

| 最好通过修改审查中的检查表项解决

示例

测试

组件

	T1	T2	T3	T4	T5
F1	X				
F2		X			X
F3			X		X
F4				X	
F5	X				X
F6		X			
F7				X	

如果确定对 F1 的修改会影响 F3，那么将重新执行测试 T1、T3 和 T5。

使用置信测试套件的选择性回归测试

选择要执行的测试子集以验证此前的功能

测试用例的审查应该包括针对置信测试套件特定测试适用性的检查表项

选择的测试涉及：

- 高频用例
- 关键功能
- 功能广度



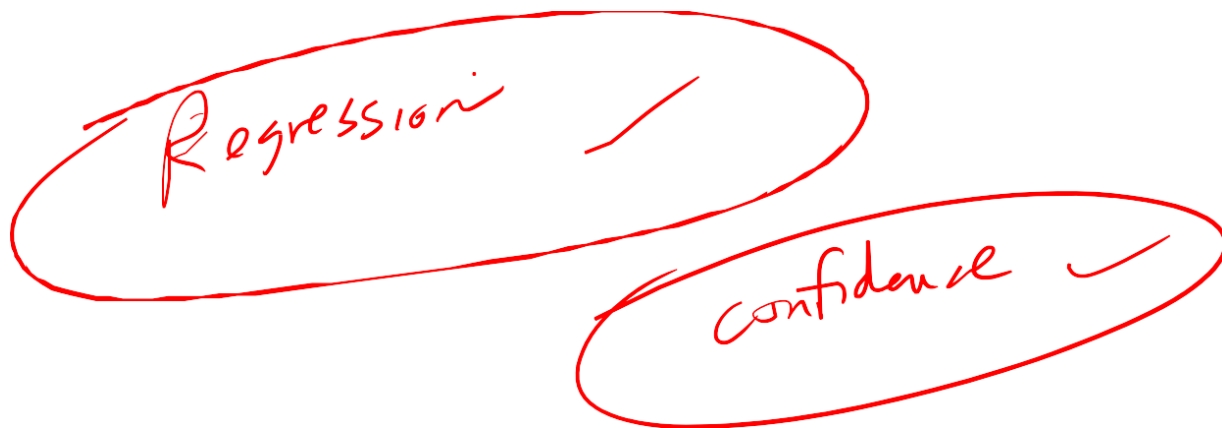
Radio on



重新验证问题

| 必须重新验证回归测试，
以确保它们与软件修改
一致

| 必须重新检查测试输入
和预期输出的正确性



总结





测试软件质量特性 –

第 2 部分

错误检测、恢复和
服务能力测试

目标



目标

开发错误检测、恢
复和服务能力测试

错误检测和恢复测试



| 系统的总体可靠性和可用性取决于系统检测各种失效和从失效中恢复的能力

- 用户
- 硬件
- 软件
- 其他系统

错误检测和恢复测试（续）

| 具有这样一个列表非常重要，它列出在需求中指定的要恢复的错误

| 一般的测试方法都包括错误注入

服务能力测试

| 对系统可用性很重要

| 目标是验证是否满足
服务能力需求

– 例如“关键问题将在 4 小时内获得修复或解决方法”

| 服务能力包括问题报告、
隔离、纠正、验证和修
复发布的所有方面

| 一般的测试方法是注入失
效和评估响应

总结





测试软件质量特性 –

第 1 部分

易用性测试

目标



目标

生成易用性测试

易用性测试

| 在许多应用程序中，
将近一半的代码都在
用户界面中

| 易用性使预期用户实现以下目标：

- 能够执行产品想要在预期环境中支持的任务
- 对他们必须遵循的规程和结果输出感到满意
- 不受其操作后果的影响

易用性需求



| 易用性需求通常用以下方面描述：

- 易学性：使用户达到期望性能水平所需的培训类型和数量
- 可记忆性：在学习产品后能够一直记得使用产品的技能
- 错误：测量用户在尝试完成任务时执行的不正确操作数
- 有效性：测量执行任务的速度
- 主观满意度：用户对产品的总体感觉

易用性测试可靠性和有效性问题

| 可靠性：如果测试是重复的，是否会得到相同的结果

- 最佳用户的速度提高 10 倍
- 25% 的最佳用户速度提高 2 倍

| 有效性：易用性测试是否测量了相关的内容

- 错误的用户
- 错误的任务

测试目标



| 形成性评估

- 了解界面的哪些方面是良好的，哪些方面是糟糕的
- 如何改进设计

| 总结性评估

- 评估界面的总体质量
- 测量测试

测试计划问题



| 用户是谁？

| 要收集什么数据？

| 他们将执行什么任务？

| 用什么标准来确定成功？

| 将提供哪些用户帮助？

试点测试



| 测试规程必须在试点研究中进行试验

| 评估

- 说明
- 成功标准
- 任务执行时间
- 评估标准

确定测试用户

| 用户必须具有
代表性

| 让新手和专家用户
一起评估

| 准备培训用户以达到专家
水平

易用性比较

| 当评估易用性选择时，
使用以下测试策略时必须格外小心：

- 受试者间测试
- 受试者内测试

| 优先选择受试者内测试

人类受试者的伦理方面

| 受试者可能担心表现不佳

| 需要让受试者感到舒适

| 强调测试系统，而不是用户

| 维护隐私问题

测试任务



| 必须具有代表性

| 一次提供一个任务

| 从简单的任务开始以增强信心

测试阶段



| 准备（确保环境设置完毕）

| 简介（欢迎、目标、概述）

| 运行测试

| 汇报结果

有声思维



| 受试者使用持续有声思维的
系统

| 测试人员可能需要定期提示
受试者

易用性实验室



| 双向镜

| 摄像机

- 用户面对面
- 用户如何与系统、文档等交互

总结





测试软件质量特性 - 第 2

部分

可靠性测试

目标



目标

应用运行轮廓测
试以评估软件可
靠性

可靠性定义（John Musa 的定义）

| “在指定时间或自然单位
数内，系统或功能在指
定环境中无失效运行的
概率”

| 自然单位对应于执行的
处理，例如完成的调用
或事务数（例如每
50,000 个事务丢失一个）

| 概率的范围介于 **0** 到 **1**

可用性定义（由 John Musa 制定）

| “在任何给定时间点，系统或系统的功能在指定环境中令人满意地运行的概率”

软件可用性计算



| 可用性也可以作为系统令人满意地运行的时间比

$$\text{可用性} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

5 个 9 可用性需求

系统应该在 **99.999%** 的时间内可用，即系统在任何时间点令人满意地运行的概率为 **99.999**

就一年中为客户提供的服务而言，这意味着每年允许停机大约 **5 分钟**

- 每年有 525,600 分钟
- 5 个 9 表示每 100,000 分钟停机 1 分钟

实现高可靠性和可用性



| 以经济有效的方式实现
高可靠性和可用性要求谨
慎应用软件可靠性工程技
术

| 必须针对以下方面应
用适当的开发技术：
- 故障预防
- 故障容错

实现高可靠性和可用性（续）



| 必须与模型一起应用适当的测试技术，以评估是否满足可靠性和可用性目标。

- 运行轮廓测试
- 错误检测和恢复测试
- 服务能力测试

运行轮廓简介

| 运行**轮廓**描述了用户如何利用产品

| 运行**轮廓**包括系统执行的一组主要功能及其发生率

| 运行**轮廓**对可靠性预测至关重要

基本运行轮廓构建步骤

1. 确定系统执行的主要功能

- 确定不同类型的用户/外部实体
- 使用用例是构建运行轮廓的良好候选项

2. 确定发生率

- 历史数据
- 市场营销

3. 计算发生概率

ATM 示例

	<u>使用用例</u>	<u>发生率</u> <u>(xact/小时)</u>	<u>发生概率</u>
存款	0.095	95	
取款		900	0.9
转账	0.005	5	

开发测试



| 根据运行**轮廓**开发测试

| 要执行的测试数量**要**基于可靠性目标

| 修改测试生成以包含具有低发生概率的关键功能

解释失效数据



| 开发测试

- 目标：为了消除导致失效的故障

| 认证测试

- 目标：为了确定应该接受还是拒绝一个软件组件或系统

运行轮廓的其他使用

| 运行轮廓还可用于：

- 指导开发的优先点
- 协助性能分析

总结





测试软件质量特性 –

第 2 部分

可靠性模型

目标



目标

确定软件可靠性
模型的工作原理

对软件可靠性增长建模

| 可靠性增长模型显示可靠性如何随时间发生变化

| 模型支持回答“何时停止测试？”问题

| 存在许多模型

| 任何可靠性测量的效率都与在测试过程中收集正确数据的效率直接相关，例如：

- 失效强度：每自然或时间单位的失效数

软件可靠性模型

| 所有模型都有假设，例如：

- 修复不会引入新的错误

| 模型通常利用数学分布来表示可靠性增长

- 泊松
- 指数

| 有效的模型预测需要使用运行轮廓进行测试

统计测试



| 测试软件的可靠性，
而不是故障检测

| 应该说明可接受的可靠性级别，并测试和修正软件直到达到该级别的可靠性

可靠性测试问题

| 运行轮廓不确定性

- 运行轮廓是否准确地反映了系统的实际使用情况？

| 测试数据生成的高成本

- 生成和检查所需的大量测试用例的成本非常高

| 高可靠性系统的统计不确定性

- 可能无法生成足够的失效来得出统计上有效的结论

增长模型选择



- | 提出了许多不同的
可靠性增长模型

- | 没有普遍适用的增长模型

- | 应对可靠性进行测量，并将观测数据拟合到几个模型

- | 可靠性预测应该使用最佳拟合模型

总结





测试软件质量

特性 - 第 2 部分

安全性测试

目标



目标

确定基本的安全性测试方法

安全性测试



| 软件正确性和安全性不是一回事

| 大多数应用程序都包含私有数据

| 安全性测试的目标是确保私有数据免受未经授权用户的访问

安全性基础



| 保密性

- 应用程序
- 数据

| 可用性

- 拒绝服务

| 完整性

- 数据修改
- 执行的功能

安全性测试上下文



| 软件可能具有
非预期或
未知功能，
将产生
副作用，
从而导致
安全性问题

| 安全漏洞要求
测试软件
与其环境
的交互

可能有安全风险软件



| 操作系统

| GUI

| 文件系统

| 其他系统

(数据库、**程序库**等)

GUI 安全风险



| 验证访问控制

- 进入系统
- 访问功能和
数据

| 评估恶意输入

- 拒绝服务

| 查找所有可能的数据访问 方法

- 剪切和粘贴
- 屏幕捕获

文件系统安全风险



| 评估如何
存储和检索数据

| 侧重于加密
和数据保护

操作系统安全风险



| 评估内存中的
解密数据
存储

| 内存不足的
压力测试

- 内存不足的
的系统可能会使数据
不受保护

其他组件安全风险



| 考虑

组件失效的结果

| 组件可能

包括程序库、
数据库等

安全性测试策略

| 拒绝应用程序访问它需要
程序库，

- 确保崩溃不会影响安全性

| 尝试通过输入长字符来
溢出输入缓冲区

| 尝试使用特殊字符
作为输入

| 尝试默认值或常见用户名和密码

安全性测试策略（续）

尝试伪造数据源

- 考虑一个具有数据包的系统，数据包通过网络发送且包含源标识符
- 伪造数据包中的源

强制系统使用默认值

- 提示时不输入数据
- 利用超时

安全性测试策略（续）



| 测试所有路径以执行任务

- 考虑打开一个文件
- 确保所有场景都经过安全性验证

| 生成每个错误消息并确保它不会威胁安全性

改进安全性测试的方法

| 查阅公共安全数据库

- CERT (www.cert.org)
- 包含有关已发布软件错误的信息

| 数据库中的有关错误的原因和**您的**产品中的可能漏洞

- 什么导致了失效?
- 在测试过程中如何检测到它?

改进安全性测试的方法



-系统容易受到攻击吗?

总结





测试管理第 1 部分

测试计划概述

目标



目标

确定系统测试计划的主要组成部分

测试计划



|所有测试层次的测试计划应为书面形式：

- 单元
- 集成（整合）
- 系统
- Beta
- 接受度

系统测试计划



|精心构想的系统测试计划是测试工作取得成功的关键

|系统测试计划须尽早开始（在软件开发需求阶段）

|系统测试计划必须体现对系统测试目标和项目限制条件的深入理解

系统测试计划（续）



|系统测试计划必须涉及：

- 系统测试目标
- 依赖性和假设
- 采用的测试策略
- 测试环境的规格说明
- 系统测试入口和出口准则的规格说明
- 进度表
- 风险管理

系统测试目标



|系统测试计划必须明确
定义系统测试活动的
目标

|可能的目标参见第 1 单元

依赖性和假设



|制定系统测试计划时，
必须确定所有**依赖性**和
假设

|示例包括：

- 资源可用性
- 软件准时完成
- _____
- _____

测试策略



| 测试策略定义如何在项目限制条件下实现测试目标

| 测试策略决定着：

- 将用于生成测试数据的技术
- 测试环境
- 入口和出口准则
- 进度表

| 测试策略通常基于风险而定

测试环境的规格说明

| 测试环境事项包括：

- 测试平台
- 模拟器
- 测试工具

| 测试环境的选择基于测试目标和选定的测试策略

- 性能测试目标可能需要负载生成工具
- 配置测试目标可能需要额外的资源和/或模拟工具

系统测试入口准则

基于测试策略制定，最大程度地提高测试效率

系统测试开始太早产生的问题包括：

- 无能力进行所有测试
- 与开发人员就问题处理进行过多交流
- 高度的重新测试

可能的入口准则包括：

- 配置管理下的代码
- 集成测试完成
- 没有未解决的高优先级问题
- 成功完成系统测试就绪度评估

系统测试就绪度评估



| 在项目早期制定，与开发一同进行

| 识别**可**有效地开始系统测试所需的功能和代码稳定性

| 为系统测试提供具体的入口准则

| 在系统测试将要开始时，**给**开发提供**一个**方法，以便确定活动的优先级

创建系统测试进度表



| 创建测试进度表需要以下活动：

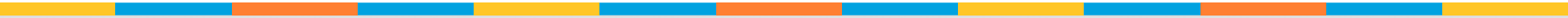
- 识别将要进行的所有测试任务
- 识别测试任务的**依赖性**
- 估算进行每项任务所需的工作和资源
- 将任务分派给个人或群组
- 将测试任务与时间线相对应

测试计划风险管理



- | 系统测试风险对应于可影响测试进度表或效率的场景
- | 测试风险可通过检查表或以往项目的“经验教训”确定
- | 测试风险须划分优先级并得到缓解
- | 优先级划分依据的是产生风险的可能性和后果
- | 风险缓解涉及降低产生风险的可能性和/或制定应急计划，以便尽量降低所发生风险的影响

总结





测试管理第 1 部分

测试进度表

目标



目标
创建测试进度
表

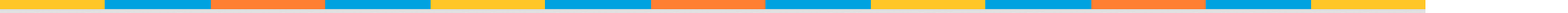
创建系统测试进度表



| 创建测试进度表需要以下活动：

- 识别将要进行的所有测试任务
- 识别测试任务的**依赖性**
- 估算进行每项任务所需的工作和资源
- 将任务分派给个人或群组
- 将测试任务与时间线相对应

测试任务示例



| 制定测试计划

| 理解需求

| 开发测试

| 审查测试

| 安装测试环境

|

|

|

分析依赖性

| 在确定任务之后，必须对测试进行**依赖性**分析

| **依赖性**可记录在 **PERT** 图中

| **PERT**（项目评估和审查技术）由海军在 **20 世纪 50 年代** 研发

| 图中网络的节点代表项目活动及其**依赖**的时间（通常为日历周），其相互关系代表优先关系

PERT 图示例



关键路径分析



| 完成项目所需的最短时间？

| 在最短时间内完成项目的关键活动有哪些？

| 哪些活动可同时进行？

| 每项活动可延迟多久但不至于影响完工日？

关键路径识别



| 关键路径是指 PERT 图经过的路径，没有富余时间

| 可通过联系每个节点、其最早的开始和完成时间来确定

| 最早的开始时间始终等于前一个节点最早完成时间的路径就相当于关键路径

估算系统测试时间



| 测试计划的重要组成部分是
估算满足测试目标所需的时间

| 估算过高会导致测试效率低下，延迟产品发布

| 估算过低会导致超时工作、
高压力和可能的低效率测试

分配任务责任



| 将类似的任务分派给同一人

| 尽量减少不必要的沟通

| 将知识和技能与任务相匹配

| 任务分配要有助于人员学习和成长

| 尝试接纳偏好

将任务与时间线相对应

| 进度必须精心安排，考虑以下因素：

- 限制条件 ✓
- 任务依赖性 ✓
- 人员的可用性 ✓
- 风险

| 进度表的制定是一个迭代过程：

- 调整任务、持续时间、资源和顺序

| 参与者必须遵循进度表

甘特图

| 进度可使用甘特图记录

| 甘特图可确定任务的持续时间及其开始和结束日期

甘特图识别同时进行的任务

| 可制作多份甘特图，展示不同程度的细节

– 甘特图的等级

甘特图示例



进度表缓冲区

| 风险管理须用于指导团队确定应急时间量（缓冲区），进度表必须分配应急时间

| 进度表的置信度与缓冲区有关

总结





测试管理第 1 部分

测试估算

目标



目标

测试工作量估算

估算系统测试时间



| 住房清洁类比

制定估算流程



| 制定测试估算中一旦确定将考虑的变量，**下一步**则须基于**这些**变量进行估算

| 基于变量制定估算的方法包括：

- 使用历史数据
- 通过成本估算模型进行计算
- 基于开发估算的百分比形成测试估算

总结





测试管理第 1 部分

测试估算步骤

目标



目标

遵循估算测试工作
量的过程

估算的重要性



| 估算是以下工作的基础：

- 测试预算
- 员工和资源分配

| 测试估算通常作为总体开发项目估算的一部分

估算不准确的主要原因



| 对需求的误解

| 缺乏估算指南

| 忽视任务

| 缺乏历史数据

| 由于时间压力，在开发估算时分析不足

| 迫于压力减少估算

通用估算过程



1. 确定估算责任
2. 评审和明确测试目标、交付物、里程碑和约束
3. 确定测试任务
4. 为测试工作选择适当的测量规模
5. 选择规模估算方法
6. 估算和记录规模
7. 估算和记录工作量

确定测试任务



| 必须定义一组测试活动

| 测试任务包括：

- 理解需求
- 工具培训
- 测试用例开发

选择适当的测量规模



| 必须利用某种类型的标准组件估算测试活动的规模

| 可能的测试测量规模包括：

- 需求数
- 使用用例/场景数
- 要测试的代码行

选择规模估算方法

自顶向下/类比

自底向上

自顶向下/类比



- | 专家基于过去的类似项目开发估算

- | 不适用于新项目

- | 如果估算人员不考虑项目之间的区别（例如专门技术、测试难度、需求稳定性等），可能会失败

自底向上估算



| 将测试工作量分解成若干部分

| 部分包括：

- 要测试的功能
- 要执行的测试类型

| 每个部分单独估算，然后对各部分求和以得出估算。

| 可以开发工作分解结构以组织各部分

估算和记录规模



- | **80/20** 法则适用于规模估算

- | 还必须传达规模估算的准确性

- | 在估算工作量之前，必须根据估算依据和所作的假设估算和记录工作规模

估算和记录项目工作量



| 必须选择工作量的规模单位

- 员工小时
- 员工月份

| 必须根据估算依据和所作的假设记录工作量估算

| 实际工作量取决于以下因素：

- 规模估算
- 估算的生产力

检查估算的策略



| 检查系统类似部分的估算是否相同

| 与其他专家一起检查

| 检查系统各部分与其他完成系统的类似部分的资源估算

总结





测试管理第 1 部分

基于风险的测试

目标



目标

执行基于执行风险的
测试

基于风险的测试策略



| 适用于有限制因素的**项目**，有必要对测试进行优先级排序的情况

| 高风险区域被认定为**这样的功能**：

- 可能出现故障
- 故障会造成严重影响

| 可用于不同的**抽象层次**：

- 子系统
- 特性
- 组件

评估故障发生的可能性



| 错误集群

| 系统的某些区域可能更容易出现错误，原因如下：

- 复杂性
- 新代码或代码变更
- 开发外包
- 历史状况不佳

评估软件故障的后果

| 应与客户和开发人员进行交流

| 可用**严重程度**来评估**职能的失效**

| 严重程度评估应关注系统的如下方面：

- 可靠性
- 可用性
- 性能
- 易用性
- 兼容性
- 维护性

| 应对需求/使用用例进行优先级排序，以便为基于风险的测试提供辅助

评估软件失效的后果



| 对于更复杂的系统，组件或功能失效后果的严重程度可能并不明显。

| 在开发人员的辅助下，可执行更严格的失效分析，可采用的评估技术包括：

- 失效树
- 失效模式影响分析

基于风险的测试策略



尽早测试高风险区域



更详尽地测试高风险区域

总结





测试管理第 1 部分

测试出口准则

目标



目标
选择测试出口准则

系统测试出口准则



| 停止测试的最佳标准是测试目标已实现之时

| 确定系统已做好发布准备的典型作法包括：

- 测量缺陷密度
- 缺陷池
- 缺陷植入
- 趋势分析
- 可靠性模型

测量缺陷密度



| 缺陷密度定义为每千条代码的缺陷数

| 过去的缺陷密度数据（若有）
可用于预测系统测试中将发现的缺陷数

| 实际数字可与预期数字作对比，以确定系统是否做好发布准备

| 这种方法存在的问题包括：

- 缺乏历史数据
- 当前系统的缺陷注入率也许不同于历史数据
- 前阶段的缺陷遏制也许不同于历史数据
- 发现的缺陷可能不会影响客户对系统的看法

缺陷池



| 适于在执行运行使用场景时使用

| 要求缺陷报告分为可单独跟踪的 2 组

| 假设每一组缺陷反映基于运行使用对整个系统的独立测试

| 可通过多种途径实施，例如：

- 将系统测试员分为 2 组
- 从 2 个独立的 Beta 测试组收集缺陷数据

缺陷池方法

| 假设以下条件进行计算：

- 缺陷_A：指 A 组发现的缺陷数
- 缺陷_B：指 B 组发现的缺陷数
- 缺陷_{A+B} 指 A 组和 B 组都发现的缺陷数

| 独特缺陷 =

$$(\text{缺陷}_A + \text{缺陷}_B) - \text{缺陷}_{A+B}$$

| 估算的总缺陷 = $(\text{缺陷}_A \times \text{缺陷}_B) / \text{缺陷}_{A+B}$

| 估算的剩余缺陷 = 估算的总缺陷 - 独特缺陷

缺陷池示例

| 假设：

- A 组发现 30 个缺陷
- B 组发现 40 个缺陷
- 20 个缺陷是 A 组和 B 组共有的

| 缺陷_A = 30

缺陷_B = 40

缺陷_{A+B} = 20

| 独特缺陷 = $(30 + 40) - 20 = 50$

| 估算的总缺陷 = $(30 \times 40) / 20 = 60$

| 估算的剩余缺陷 = $60 - 50 = 10$

缺陷植入

| 存在争议的方法涉及将缺陷“植入”系统

| 假设缺陷可植入系统中，
这些缺陷代表客户将遇到的缺陷

| 假设发现剩余缺陷的能力相当于发现所植入缺陷的能力：

- 估算的总缺陷 = (植入的缺陷 / 发现的植入缺陷) x (发现的正常缺陷)

缺陷植入示例

| 假设：

- 植入 20 个缺陷
- 测试发现 10 个植入的缺陷
- 测试另发现 50 个缺陷

| 估算的总缺陷 = $(20 / 10) \times 50 = 100$

| 估算的剩余缺陷 = $100 - 50 = 50$

趋势分析



| 可通过分析趋势数据来得知软件的就绪度：

- 失效时间
- 累积失效数
- 每时间单位的失效数（失效强度）

| 可识别三个趋势中的一个：

- 可靠性降低
- 可靠性提高
- 可靠性稳定

失效时间



累积失效数



每时间单位的失效数



趋势分析（可靠性降低）



| 预计在新活动开始时进行

- 新测试阶段
- 新的测试类型
- 不同的用户概况

| 时间长预示可能存在严重的
软件问题

趋势分析（可靠性增加）



| 通常是好消息

| 但突然增加可能是因为如下原因 导致：

- 测试投入改变
- 测试倦怠
- 失效情况未记录

总结





测试管理第 1 部分

测试文档

目标



目标

识别各种不同类型的
测试文档

IEEE 测试文档



| 参见 IEEE STD-829
IEEE
软件测试文档标准

| 提供的模板适有：
-测试计划
-测试用例
-测试事故报告
-测试总结报告
-其他测试文档

测试用例



| 测试用例规格说明标识符

| 测试执行环境

| 测试项

| 遵循的特殊测试流程

-用于产生测试的文档参考文献

| 测试用例相关性

| 输入规格说明

| 输入规格说明

测试事故报告



| 测试事故报告标识符

| 总结

| 事故描述

- 输入
- 预期结果
- 实际结果
- 日期和时间
- 环境
- 重复次数

| 影响

测试事故报告



| 必须仔细撰写

| 目标受众包括：

- 程序员
- 其他测试员
- 经理
- 质量保证
- CCC 委员会

| 必须提供足够的信息，以便
每位查阅者完成他们的工作

开发人员心态



| 测试员需了解程序员的工作任务

- 复杂任务
- 模糊不清且不断改变的需求

| 大部分开发人员只负责系统的一个部分

- 测试员是通才
- 测试员须帮助开发人员了解全局

严重程度与优先级



| 测试人员需认识到严重程度与优先级之间的差异

| 严重程度反映错误对客户的影响

| 优先级体现项目的考虑事项

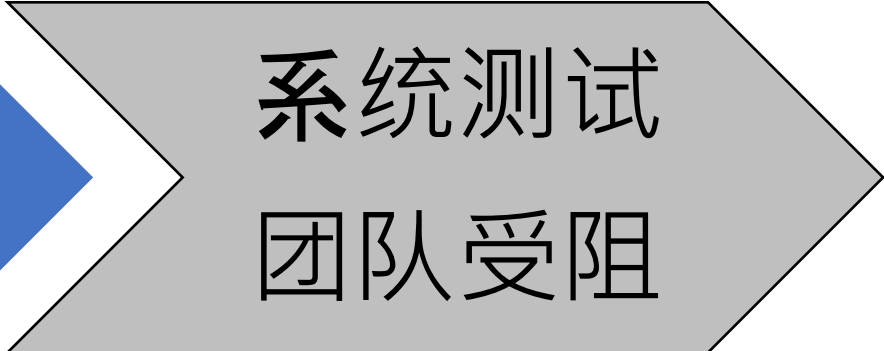
| 并非所有非常严重的错误都属于高优先级

| 并非所有高优先级的错误都很严重

系统测试问题优先级



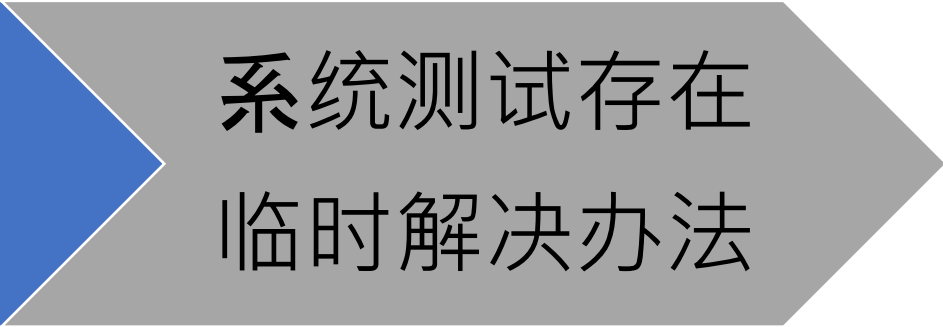
A :



系统测试
团队受阻



B :



系统测试存在
临时解决办法

审查缺陷报告？

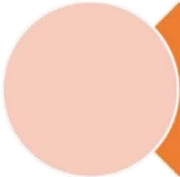


| 某些组织为了节省时间，会让另一位测试人员审查缺陷报告。

| 审查标准包括：

- 完整性
- 可重复性
- 清晰度
- 严重程度评估

测试总结报告



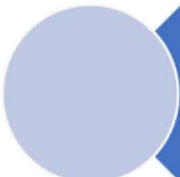
测试内容总结



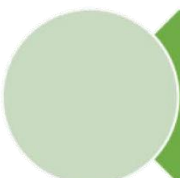
方差



综合评估



结果总结



评估

测试文档



**| 敏捷开发方法迫使重新
思考文档需求**

**| 测试文档的水平 and 格式必
须基于：**

- 文档如何支持测试活动
- 文档是否为交付物
- 文档监管是否受到关注
- 文档是否支持跟踪活动
- 支持回归测试

总结





测试管理第 2 部分

测试跟踪

目标



目标

利用各种措施跟踪测试进度。

系统测试跟踪

| 在系统测试期间，必须收集和跟踪信息，以评估：

- 产品质量（之前在“系统测试出口准则”中讨论过）
- 测试进度

| 依据计划跟踪测试进度的措施有很多，包括：

- 已开发测试的百分比
- 已执行测试的百分比
- 已测试需求的百分比

| 测试进度表和工作量进度也可以通过挣值来评估

挣值

| 挣值是一项跟踪进度表和成本进度的技术

| 挣值**方法**为每项任务建立一个相对值，并在任务完成时计入该值

| 然后根据以下方面跟踪进展：

- BCWS (计划工作的预算成本)
- BCWP (完成工作的预算成本)
- ACWP (完成工作的实际成本)

挣值示例

假设我们正在测试一个产品的两个同时开发的增量版，增量版分别标为“1”和“2”，并且每个增量版都有标为“A”-“F”的特性。因此，任务“1A”对应于增量版“1”中测试特性“A”的活动。与每个测试任务相关联的是对应于要完成任务的估计工作量的挣值。

任务	挣值 EV	任务	挣值 EV
1A	50	2A	30
1B	40	2B	40
1C	30	2C	30
1D	20	2D	50
1E	50	2E	40
1F	30		

挣值示例

(续)

假设以下进度表：

第 1 周	第 2 周	第 3 周	第 4 周
1A, 1B	1C, 1D	1E, 1F	
	2A, 2B	2C, 2D	2E

挣值示例

(续)

周	已完成工作	成本
1	1A, 1B, 1C	100
2	1D, 2A, 1E	70

挣值示例

(续)

	第 1 周	第 2 周	第 3 周	第 4 周
BCWS	90	210	370	410
BCWP	120	220		
ACWP	100	170		

挣值示例

(续)

| 基于挣值，该项目在第 2 周结束时提前完成

BCWS = 210

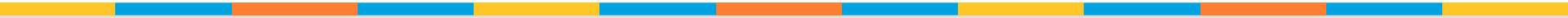
BCWP = 220

| 基于挣值，项目在第 2 周结束时低于预算

BCWP = 220

ACWP = 170

总结





测试管理第2部分

测试过程改进

目标



目 标
探索改善
测试过程
的策略

改善测试过程



| 软件测试团队必须不断寻找更快，更有效的测试方法

| 过程改进阶段

- 表征当前过程
- 分析当前过程
- 表征目标过程
- 过程重新设计
- 实行

表征当前的过程

| 了解当前的测试
过程很重要

| 必须谨慎区分以下各项：

- 感知过程：您认为你该做的工作。
- 正式过程：您应该做的工作。
- 实际过程：您的工作。

引用沃茨·汉弗瑞（Watts Humphrey）



“如果您不知道自己在哪里，地图将无济于事。”

分析当前过程



| 辨识增值和非增值活动

| 辨识过程中的变异的来源

| 软件度量指标在客观的过程分析中扮演关键角色

| 软件测试领导人必须计划收集适当的度量指标

测试度量指标



| 你无法控制无法衡量的事情

| **GQM模型**（目标-问题-度量指标）

- 定义测量过程的目标
- 导出为实现目标必答的问题
- 制定度量指标以回答问题

测试目标示例



| 减少测试时间

| 找出更多严重的缺陷

测试问题示例



| 在测试期间，我们大部分时间都花在哪里？

| 我们遗漏哪些类型的缺陷？

| 我们的测试生产力是多少？

可能的测试度量



| 测试生产力

| 测试质量

表征目标过程

| 辨识过程的最终目标，例如

- 减少测试时间
- 找出更多高度严重性的问题

| 引用沃茨·汉弗瑞

(**Watts Humphrey**)

- “如果您不知道要去哪里，那么选任何一条路都可以。”

过程重新设计



| 朝目标过程的方向改进
当前过程

| 探索各种消除、简化
或合并活动的方法

| 探索各种消除重复
工作的方法

| 探索各种减少任务变数
的方法

实施过程改进的元素



| 设定过程改进目标

| 衡量实现目标的进
度

| 实施过程改进的元素

- 从定义明确的候选项开始
- 对未经过验证的候选项进行试验性的测试

过程改进工具 - 事后（验尸）检查 / 所学的经验与教训 / 回顾



| 经验与教训要存档记录

| 分析技术包括：

- 与关键人员访谈
- 数据统计分析
- 重大问题调查
- 指出过程中进展顺利和失败的地方

测试所学经验教训讨论



| 总体进度分析

| 整体质量

| 入口条件标准的适合性

| 团队互动

| 出口条件标准的适合性

示例问题的讨论



| 针对发展的沟通和协作效果如何？

| 测试评估的效果如何？

| 测试环境是否有任何问题？

| 我们遗漏了哪些类型的缺陷？

| 我们在哪里浪费时间？

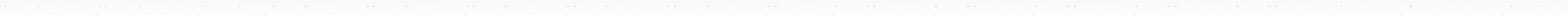
过程改进工具-因果分析



| 寻找缺陷的根本原因以
及防止未来发生的方法

| 在另一个模块中详述

总结





测试管理第 2 部分

测试外包

目标



目标

制定外包测
试计划

家庭类比



| 你是自己做这项工作还是雇一个承包商来为你做这项工作？

影响外包决策的因素



| 对专业/先进技术或能力的需求

| 系统的战略价值

| 成本

| 战略联盟

| 维护支持

| 开发速度

| 对降低人员配备的渴望

测试外包活动



| 定义要分包的工作

| 制定外包计划

| 如果需要，执行域和/
或过程评估

| 选择分包商

| 与分包商的
合同

| 监督测试，以确保它是按计划进行的、在成本范围内，并满足需求

| 接受测试

定义分包工作

| 为产品及其活动分区

| 选择分包工作以：

- 将机构的效率最大化
- 匹配潜在分包商的技能和能力
- 尽量减少沟通和协调工作量
- 尽量减少依赖性
- 将分包商的知识未转移到机构的风险降至最低

制定分包商管理计划

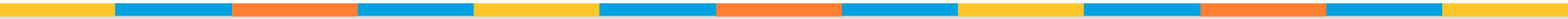


| 为工作制定技术规格说明

| 为供应商制定工作说明书

- 识别将要开展的所有任务
- 识别要遵循的相关过程
- 识别维护责任

制定分包商管理计划（续）



| 执行风险管理

- 尽早识别外包风险
- 调整购买策略，将风险降至最低

| 估计所需资源

- 评估供应商的测试工作量（成本）
- 评估厂商的工作量

外包协议的组成部分



| 合同

| 技术规格

| 工作说明书

供应商管理活动



| 选择

- 搜索和评估供应商
- 规定合同的法律条款
- 谈判合同

| 分包管理

- 传达和解释需求
- 监控供应商，包括评审
- 解决问题

选择供应商时要考虑的因素



| 战略商业联盟

| 可用员工

| 类似工作的前期表现

| 领域专业知识

| 地理位置

| 成本

| 软件测试和软件管理能力

| 过程和工具的相似性

| 商业可行性

与供应商的合同

| 见律师！

跟踪和监督

| 必须为供应商跟踪和监督制定计划

| 计划必须解决：

- 要执行的活动以及执行这些活动的时间表
- 工作组的识别、分配的责任和组间的沟通
- 用于**评审**和跟踪供应商绩效的技术、工具和方法
- 上报程序

跟踪和监督活动



| 保持良好的沟通

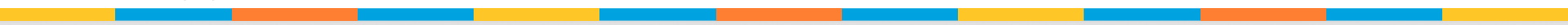
| 持续风险管理

| 评审

| 发票审批

| 度量

工作验收



| 工程验收是一个涉及
合同的正式程序

| 你必须确信测试是彻底和
完整的

总结





测试管理第 2 部分

人员管理

目标



目标

应用最佳实践
来最大限度提
升测试团队的
绩效

利用资源杠杆以改进软件开发



有效地利用人员



| 项目取得成功的最重要因素之一是人员的能力、经验和动力

| 软件测试领导必须尽一切可能最大限度地提高个人和团队的效率

系统测试领导的职责



| 与项目管理**人员**协商进
度和资源

| 与开发团队就入口准则
进行协商

| 确定测试团队的优先级

| 管理团队

基本管理技能



| 领导力

| 协商

| 沟通

| 激励

| 委派

| 解决问题

激励



哪些因素可激励软件测试员？

领导者激励指南

| 了解你团队的需求、目标和动机是什么

| 提供满足人员需求的工作环境和任务结构

| 实施公平的奖励制度

- 认可、技术进步、技术知识和责任增长以及薪水都是重要因素

| 如果团队清楚地传达并了解了目的、计划、要求和预期，则团队成员将受到激励去执行任务

软件测试员的重要激励因素



| 工作自豪感

- 以自己工作为荣的测试员的工作满意度和质量会增加
- 测试组长必须确保测试员对自己的工作“感到自豪”
- 测试组长必须确保测试员能够完成自己的任务

软件测试员的重要激励因素



| 成就自豪感

- 早期取得成功会增加动力
- 整理安排项目活动以实现早期“成就”

| 贡献自豪感

- 人们有一种强烈的贡献欲望。
- 认识到每个测试员贡献的重要性至关重要

结对测试



- | 涉及 2 位测试员探索系统输入空间以执行测试

- | 类似于极限编程中的结对编程

- | 强制进行测试策略的沟通和分类

- | 利弊

团队发展



| 提升所有团队成员的
知识和技能很重要

| 团队发展策略包括：

- 培训
- 轮岗
- 辅导
- 评审、评估和反馈

团队工作的重要性



| 项目取得成功需要掌握技能的人员作为一个团队有效协作

| 团队的成功由明确的项目目的沟通驱动

- 团队必须了解要实现的目标
- 团队成员个人必须了解自己如何为团队的目标做出贡献

优良团队的特征

(选自 Paulish 的《软件项目管理实用指南：以体系结构为中心》(Architecture – Centric Software Project Management))



| 如果有团队成员遇到困难，其他人自觉帮忙

| 团队互动“轻松”

| 冲突很快得到解决

| 团队成员相互了解，能够预测大家对提出想法的反应

| 团队成员全心全意投入项目

优良团队的特征（续）

- | 团队成员在讨论成就和问题时，使用“我们”而不是“我”
- | 团队领导更多地被视为“教练”而不是“独裁者”
- | 团队成员了解他们的角色和责任
- | 团队为自己的工作感到自豪
- | 团队从错误中吸取教训
- | 团队相信自己会取得成功

总结





测试管理第 1 部分

测试审查

目标



目标

计划并参与各种
工作产品审查

系统测试员参与审查

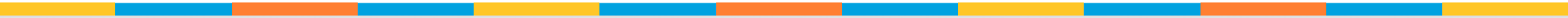


| 系统测试员应在开发过程中
参与需求文档审查

| 系统测试员还应审查其
工作产品：

- 测试计划
- 测试用例
- 测试事件报告

定义



|审查是一个“用于评估文档及其生产的正式统计过程控制方法”

|审查需要完成以下工作：

- 提前准备
- 利用规则和**核对表**
- 度量收集和分析，以促进过程改进

审查包



要审查的工作产品



支持文档



核对表



审查日程

核对表

| 审查员的
“工作辅助工具”

| 审查过程中的基本工
具

| 指导缺陷检测过程

| 按照项目和文档自
定义

| 以电子形式提供

可测试性问题

| 测试员必须了解可测试性特性

| 可测试性同时涉及可见性和可控性

| 示例包括：

- 诊断
- 在执行过程中查看或修改数据的测试点
- 能够将系统置于给定状态

需求核对表

1.

2.

3.

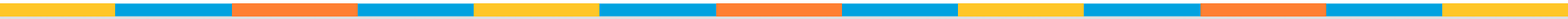
4.

5.

6.

7.

测试计划核对表



- 1. _____
- 2. _____
- 3. _____
- 4. _____
- 5. _____
- 6. _____
- 7. _____

测试用例核对表

1.

2.

3.

4.

5.

6.

7.

总结





测试管理第 2 部分

因果分析

目标



目标

应用因果分析
来提高测试有
效性

因果分析



| 1983 年由 IBM 首次引入

- IBM 报告称，不到 1% 的因果分析投资使错误率降低 50%

| 力求确定缺陷的根本原因以及避免未来出现缺陷的方法

| 收集和分析缺陷数据

因果分析步骤



选择一组测试遗漏的缺陷进行分析



对于每个缺陷，确定未检测到缺陷的可能原因



确定缺陷的常见原因



确定消除常见缺陷原因的可能解决方案

缺陷原因类别



沟通问题

| 由于缺少信息或信息不正确而遗漏缺陷

| 示例：

- 需求文档相关的问题
- 最后一刻的需求变更未传达给测试团队

缺陷原因类别（续）



疏忽

| 由于测试时未能考虑所有用例或组合而遗漏缺陷

| 示例：

- 未能测试特性的相互作用
- 未处理所有输入组合
- 未测试边界值

缺陷原因类别（续）



培训

| 由于缺乏对测试方法或工具的了解而遗漏缺陷

| 示例：

- 实验设计应用不正确
- 测试员不了解如何生成自动化测试

缺陷原因类别（续）



记录

| 因为简单的错误遗漏
缺陷

| 示例：

- 测试员未能仔细比较预期结果
与实际结果

确定可能的根本原因



五个为什么

| 足够多次的询问为什么以找到缺陷的根本原因

| 最多可能需要重复 5 次

示例



| 考虑客户报告的特性
相互作用错误

| 为什么未在测试期间检测到缺陷

- 从未测试过特性相互作用

| 为什么未测试特性的相互作用

- 未与测试团队沟通后期特性添加

制定可能的解决方案



疏忽问题

| 使用**核对表**

| 自动**核对**工具

| 工作产品模板

| 审查

制定可能的解决方案（续）



培训问题

- | 即时培训

- | 教程

- | 合理的人员配备

制定可能的解决方案（续）



沟通问题

- | 与其他小组的联络

- | 变更跟踪系统改进

- | 已改进文档

- | 变更过程

制定可能的解决方案（续）

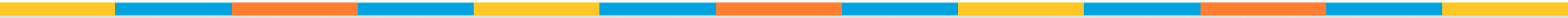


记录问题

| 自动化工具

| 审查

总结





测试管理第 2 部分

测试成熟度模型

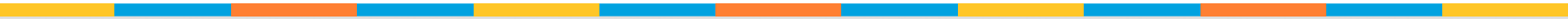
目标



目标

评估测试机构的
成熟度等级。

适用的成熟度模型



CMMI

TMMI

CMMI 等级



| 初始级：adhoc

| 可重复级：依赖个人的专业知识

| 已定义级：已定义并记录过程

| 已管理级：度量广泛用于指导过程

| 优化级：注重持续改进

TMMI



| 以类似方式开发，**用来评估**
测试过程的成熟度

| 已定义成熟度的五个等级

| 每个等级都有通过活动、
任务、职责 (ATR) 实现的
目标和子目标

TMMI 等级 1：初始级



| 随机测试，无特定目标

| 测试没有优先级

| 在编写代码后开始测试过程

| 未执行测试跟踪

TMMI 等级 2：阶段定义级



| 已开发软件测试计划，其中
包括明确定义的测试阶段

| 已引入基本的测试方法
| 已定义测试目标

TMMI 等级 3：集成级



| 软件测试集成到软件开发过程

| 测试进度得到监控和跟踪

| 测试员经过培训

| 已执行测试计划风险管理

TMMI 等级 4：管理和度量级



| 已引入评估测试过程的度量

| 已引入审查过程**来**评审有效性和效率

TMMI 等级 5：优化级，缺陷预防和质量控制级



| 已执行根本原因分析，
防止缺陷流向客户

| 统计质量控制用于监控测试过程

| 已实现测试过程改进

总结

